

Abstract

The increasing prevalence of Internet of Things (IoT) technology is also leading to a rise in potential cybersecurity risks. Therefore, it has become crucial to develop effective solutions using deep learning to enable us to discover the various vulnerabilities that may arise from continuous technological advancements, and to defend against the different types of attacks that fraudsters might use to exploit system weaknesses, such as DDoS attacks, web attacks, and spoofing. Most model failures stem from their reliance solely on detecting spatial patterns and their failure to focus on detecting temporal patterns, resulting in poor performance in detecting temporal patterns that evolve over time, thus making their detection more difficult. This study proposes an advanced framework based on the Sliding Window [1] technique to collect 10 consecutive records, enabling deep learning models to extract hidden time patterns between attacks. This framework was evaluated on five robust models: AE-LSTM-CNN [2], CAE-TCN [3], CNN-LSTM [4], LSTM [5], and CNN-BILSTM [6]. These five robust models were fed onto the CIC-IoT-2023 [7] dataset, addressing the inherent imbalance using a hybrid approach combining Under Sampling using RandomUnderSampler [8] and SMOT [9]. The dimensions of 28 features were reduced to minimize noise and scattering using the RandomForest [10] algorithm. The experimental results showed the superiority of the LSTM [5] model, achieving the highest accuracy at 98.58% and an F1-Score of 98.00%. This was followed by the CAE-TCN [3] model, which demonstrated an excellent balance between accuracy (98.56%) and training speed. These results suggest that using Sliding Window [1] technology with cached deep learning models enhances their ability to detect time-period patterns between attacks, thus facilitating the detection of small-footprint attacks on the network.

1 Introduction

The development in the Internet of Things (IoT) has far surpassed traditional technologies, as this field offers a genuine addition to people's lives by improving their quality of life. Many people's lives depend on IoT technologies, as it provides solutions in smart healthcare, smart transportation, and home automation. Therefore, the Internet of Things is one of the most widespread fields, with the number of devices falling under the IoT umbrella expected to reach 125 billion by 2030.

However, despite its immense importance, the Internet of Things (IoT) remains a double-edged sword if not properly secured. It can pose a real threat to all aspects of our daily lives if robust security measures are not implemented, including securing the physical devices themselves, the software that controls them, and the cloud storage of data. Among the most prevalent threats in this field are DDoS attacks, which can lead to a complete denial of service, and web-based threats, which can result in the leakage of sensitive user data, as well as many other threats that we will discuss later.

Traditional systems typically face significant challenges in securing IoT devices. Fortunately, Intrusion Detection Systems (IDS) have witnessed significant advancements and now perform exceptionally well with the development of artificial intelligence technologies, particularly deep learning. Despite this considerable progress, significant research gaps remain. Most research focuses on evaluating most traffic as isolated events, which makes it difficult to detect modern attacks that leave a small footprint on the network. This makes them harder to detect because these attacks evolve significantly over time, rendering a focus solely on spatial features largely ineffective. Temporal features must also be considered.

To address this challenge, this research presents an advanced framework focused on extracting temporal features using the Sliding Window [1] Technique. This involves collecting 10 consecutive sequences of records and then feeding them to the model, enabling it to detect temporal patterns between attacks. This allows the model to accurately distinguish attacks with small footprints that change over time. We tested and compared this technique on five highly robust models: AE-LSTM-CNN [2], CAE-TCN [3], CNN-LSTM [4], LSTM [11], and CNN-BILSTM [6]. These models were rigorously evaluated using the highly robust and realistic CIC-IoT-2023 [7] dataset.

In these five models that we compared, the researchers faced three main challenges.

- The gap between spatial and temporal features: This challenge has posed a problem for researchers, as most modern web attacks are characterized by small, lightweight network footprints and evolve slowly over time, making researchers' reliance on spatial features alone insufficient.
- The problem of class imbalance: In modern datasets such as CIC-IoT-2023, normal traffic or DDoS attacks overwhelm other types, leading to model bias and failure to detect rare and critical attacks.
- The need for hybrid models: Recent research, such as AE-LSTM-CNN [2], CAE-TCN [3], CNN-LSTM [4], CNN-BILSTM [6], has shown that single models alone do not achieve a sufficient balance between speed and accuracy. This necessitates a shift towards hybrid models that achieve a balance between accuracy and speed. Some of these models work to detect spatial features, such as CNN, while others work to detect temporal features, such as LSTM and TCN, which are characterized by their strength in detecting temporal patterns.

The AE-LSTM-CNN [2] model is based on a hybrid architecture operating in parallel, consisting of an autoencoder (AE), long-term memory (LSTM), and a convolutional neural network (CNN). This model leverages the unique characteristics of each module and then integrates them. The autoencoder is used to extract spatial features and compress data to reduce noise. The long-term memory is used to extract and detect temporal patterns, utilizing the module's internal cache to store attacks, and then to detect patterns. The convolutional neural network is used to detect spatial patterns and perform the final classification. After training, this model achieved 98.44% accuracy, 97.80% precision, 97.79% recall.

The CAE-TCN [3] model is based on a hybrid architecture that utilizes a convolutional autoencoder (CAE) and a time convolutional network (TCN). The CAE acts as the autoencoder, compressing data properties to reduce noise, while the TCN is used to detect time patterns. This model performed well after training, achieving accuracy of 98.56%, precision of 97.99%, and recall of 97.96%.

This CNN-LSTM [4] architecture is considered a classic approach to Intrusion Detection, where the CNN layer relies on spatial feature detection and the LSTM layer relies on temporal feature detection. After completing the training process for this model, it achieved the following results: Accuracy 97.02%, Precision 95.89%, Recall 95.82%.

This architecture LSTM [11] is based on the LSTM model, which contains temporary storage cells that enable it to retain attack data and detect time patterns. This aligns with our research methodology, which utilizes the Sliding Window [1] technique. This technique involves collecting 10 attacks and then sending them to the model during training to enable it to detect hidden time patterns between attacks. This architecture achieved the best performance in detecting time patterns and in its compatibility with the Sliding Window [1] technique, achieving accuracy of 98.58%, precision of 98.02%, and recall of 9.99%.

This hybrid CNN-BILSTM [6] architecture relies on two powerful models: CNN and BILSTM. The CNN model is used to extract spatial features, while the BILSTM model is used to extract temporal features from both directions. This dual capability enables the model to detect temporal patterns from both the front and back. This architecture has achieved high performance, with accuracy rates of 98.14%, precision of 97.35%, and recall of 97.36%.

We used the Sliding Window [1] technique, which is the core feature of our research. Through this technique, we tested the ability of these five models to detect time patterns. This technique aggregates consecutive records (which we defined as 10 records in this research) and converts them into a single record of a fixed length. This allows the models with internal cache memory to detect hidden time patterns between attacks, giving them a high capacity for reading and classifying subtle attacks that leave a small network footprint.

In the remaining sections of the research, we present the detailed research methodology. Section 2 outlines the chosen methodology, and Section 3 presents the results and analyses generated from the training process for the five models.

2 Related Works

In this section, we will review all previous work in Intrusion Detection and briefly present the benefits of this work and the gaps that were not addressed. To overcome the problem of constantly retraining models when new threats emerge, Park et al [12]. proposed Siamese neural networks (CNNs) and a few-shot learning technique by converting textual data into 3D images. This resulted in a 3% improvement in accuracy and up to a 6% increase in the recall rate. Despite the model's efficiency in reducing training time, the researchers indicated the need to improve the model's hyperparameters to enhance its ability to detect entirely new threats and internal cyberattacks, in addition to the necessity of testing it on more up-to-date databases. Halbouni et al [4]. presented a hybrid model combining CNN and LSTM to detect both spatial and temporal features, achieving impressive performance of 99.67% and 99.64% on the WSN-DS and CIC-IDS 2017 databases, respectively. However, the researchers acknowledged a problem with the model's ability to detect small data sets due to data imbalances. Neto et al [13]. presented a comprehensive study combining 59 scientific papers in which they classified deep learning techniques used in emerging systems such as edge computing and the Industrial Internet of Things (IIoT). Despite this, the researchers discovered a significant gap between the model's performance in the lab and its performance in a real-world IoT environment. Alsubaei [14] developed an intelligent intrusion detection system based on improved preprocessing, combining the XGBoost and OSNN models. This system achieved exceptional accuracy of 99.93% on NSL-KDD data and 99.53% on CICIDS2017 data. However, the researcher noted that results achieved on standard databases may not reflect the full diversity of real-world Internet of Things (IoT) environments. Wu et al. [15] proposed three stacked models, StackMean, StackMax, and StackRF, with the use of SMOTE to address data imbalances. KPCA for nonlinear feature extraction achieved superior results in detection accuracy and sensitivity on the WUSTL-EHMS-2020 dataset. The researchers noted that the current model is centrally trained, highlighting the future need to integrate it into federated learning frameworks. Wu et al. [16] presented three advanced stacked models (StackMean, StackMax, and StackRF) in the context of the Industrial Internet of Things (IIoT), using SMOTE technology for data balancing. These structures achieved exceptional accuracy of 99.99% on WUSTL-IIoT-2021 data and 99.81% on NSL-KDD data. Despite the effectiveness of the proposed models, the researchers noted a trade-off: the use of SMOTE increased the recall rate for rare attacks at the expense of increasing false alarms. Researchers conducted a comparative study [17] between traditional machine learning algorithms (RF and DT) and deep reinforcement learning (DQL) for intrusion detection. The results showed that Random Forest (RF) outperformed other algorithms in detecting known attacks with an accuracy of 99.95%, while the DQL model

demonstrated superior generalization capabilities and zero-day attack detection with an F1 score of 90.28% across three datasets: CIC-IDS2017, NSL-KDD, and AWID. Despite its flexibility, the DQL model suffers from slow inference time and high computational training costs. Ali and Al-Sharafi's [18] study focused on detecting and mitigating man-in-the-middle (MitM) attacks in IoT networks by comparing the performance of LSTM, Random Forest, and SVM models. The results showed that the Random Forest model outperformed others, achieving 94% accuracy and a recall rate of 0.864. Despite the model's success, the researchers noted that deep models like LSTM still suffer from low accuracy and a high false alarm rate. Reddy et al. [19] proposed a new framework called the 'Twined Ensemble model', which selects a pair of algorithms (from Random Forest, AdaBoost, and Gradient Boosting) specifically for each attack class based on its individual performance. Despite the outstanding results, the researchers noted that relying on the Gradient Boosting algorithm increases computational cost and training time (428.57 seconds), which could hinder deployment in resource-limited environments. Shukla et al. [20] presented a sophisticated hybrid model integrating optimization and deep learning algorithms. They used KPCA for dimensionality reduction, followed by a Grasshopper Optimization Algorithm enhanced with Lévy flight to automatically fine-tune the hyperparameters of the DNN-LSTM model. The results demonstrated the superiority of this approach in improving detection accuracy, particularly in novel and small-sampling attack scenarios. However, the researchers noted that current systems still face challenges in adapting to the constant changes in traffic behavior caused by internal and external influences, as well as the problem of false alarms resulting from data imbalances in IoT protocols. Qiu et al. [21] proposed FedAware, a federated learning framework that integrates an auto-shrinking encoder and a single-class classifier (OC-SVDD) to improve anomaly detection. The results demonstrated FedAware's superiority in accuracy and stability. Despite FedAware's efficiency in simulated environments, the researchers acknowledged the need to test the system in complex, real-world scenarios. Buyuktanir et al. [22] presented a comprehensive study on integrating federated learning (FL) into intrusion detection systems (IDS), reviewing the ability of algorithms such as FedAvg, FedProx, and FedYogi to achieve accuracy of up to 99.27% across diverse datasets such as Edge-IIoTset and UNSW-NB15. Despite this remarkable progress, the researchers identified software and organizational gaps in how to manage untrusted participants and balance the trade-off between computational costs and communication overhead. To address the complexities of distributed networks and ensure data privacy, researchers [23] proposed a hybrid model (CRNN) that combines convolutional neural networks (CNNs) for spatial feature extraction and recurrent neural networks (RNNs) for temporal pattern recognition, within a federated learning framework. The model achieved excellent accuracy of 98.93% on the Edge-IIoT dataset. Despite this success, the researchers acknowledged that the high computational

cost of training and optimizing this hybrid model may hinder its real-time application on resource-limited Internet of Things (IoT) devices. Zahid and Bharati [6] proposed a hybrid deep learning model that combines convolutional neural networks (CNNs) for accurate spatial feature extraction with bidirectional short-term memory networks (Bi-LSTMs) for capturing temporal dependencies, achieving 99.9% accuracy on KDDCup99 data, 99.8% on NSL-KDD, and 98.0% on CICIDS2017. Despite the high accuracy of the proposed model, the researchers emphasized the need for future testing on dedicated and updated databases for Internet of Things (IoT) environments (such as TON_IoT and Bot-IoT) to ensure its effectiveness in real-world attack scenarios. Wang et al. [3] proposed an incremental learning framework called CTWA, which combines a convolutional autoencoder (CAE) and a time convolutional network (TCN) to extract spatiotemporal features. It achieved 96.43% accuracy on the CIC-IoT-2023 dataset and demonstrated a strong ability to detect unknown attacks. Despite the model's classificatory power, its training time (789.58 seconds) is high compared to competing models, potentially limiting its use in systems requiring immediate updates. Susilo et al. [24] proposed a three-stage hybrid framework called AE-LSTM+CNN for securing IoT environments. The strategy relies on using Automated Encoders (AEs) to extract robust features from unstructured data, followed by LSTMs to analyze temporal patterns, and finally CNNs for the final classification of attacks. The system demonstrated high efficiency in improving detection accuracy and reducing false positives on the CIC-IoT-2023 database. The researchers acknowledged that using SMOTE might introduce biases that do not accurately represent the true attack behavior; therefore, the study recommended moving towards more advanced generative technologies such as GANs to generate synthetic data that incorporates temporal and spatial dependencies. Sajid et al. [25] proposed an advanced hybrid intrusion detection model based on two strategies for handling diverse datasets. They used XGBoost-LSTM for UNSW NB15 and NSL KDD datasets, and CNN-LSTM for CIC-IDS 2017 and WSN DS datasets. Experiments showed that using XGBoost as a feature extraction tool before classification via LSTM performed slightly better than CNN-LSTM. Despite the model's efficiency, the researchers noted that the complexity resulting from combining different models led to a higher training time compared to individual models, which could be a limitation in resource-constrained environments. The study by Halbouni et al. [26] presented a hybrid model combining Convolutional Networks (CNN) and Lastom Recursive Networks (LSTM) to exploit the former's ability to extract spatial features and the latter's ability to analyze the temporal features of network movement. This model achieved results of 99.64% on CIC-IDS 2017 data and 99.67% on WSN-DS data, outperforming individual models. Despite the overall strong performance, the researchers acknowledged the model's weakness in detecting certain rare types of attacks such as (Web attacks, Worms, and Backdoors), which they attributed to the problem of Data Imbalance. This study [5]

reviewed the performance of a range of deep learning models (1D CNN, LSTM, RNN, MLP) in real-time threat detection using the comprehensive CIC IoT-DIAD 2024 database. The results demonstrated the superiority of the 1D CNN model as the best intrusion detection algorithm, achieving 99.53% accuracy in binary classification and 99.12% in multi-class classification. The study highlighted significant challenges in the real-world deployment of these models, most notably the 'black box' problem, which hinders understanding the decision-making process in deep learning. It suggested integrating explainable artificial intelligence (XAI) techniques to enhance transparency.

3 Methodology

In this section, we present the complete research methodology we adopted for the Dataset, the operations we performed on it, the methods of handling it, the models we chose and their architecture, the data splitting for training and testing the models, and their illustrative diagrams.

3.1 Dataset

This research utilizes the CIC-IoT-2023 [7] dataset, a flow-based IoT intrusion detection dataset. This dataset includes 33 different attacks, which were subsequently grouped into 7 main categories: DDoS, DoS, Miri, Recon, Spofing, Web, and Brute Force, as shown in Table 1. This dataset was collected in a simulated real-world IoT environment using 105 real devices. Malicious and benign attacks were realistically represented in this environment, making this dataset an excellent example of a realistic IoT environment due to its detailed and accurate representation of 33 different attacks. This dataset was produced by the Canadian Cyber Security Research Institute (CIC). Given the institute's credibility and commitment to publishing accurate scientific research in the field of cybersecurity, we used approximately 15% of the data, as shown in Table 2. This table provides a detailed description of the amount of data used in each category before any modifications were made, Although datasets are a true treasure, they have some drawbacks, such as their limited realism and scalability. They simulate a fixed set of devices and lack the evolution, dynamic characteristics, and noise found in natural IoT environments. Another problem is that datasets suffer from a strong bias towards larger categories like DDoS attacks compared to smaller categories like Web and Recon attacks. This can lead to problems in model generalization, as the model may favor larger categories because they occur in massive quantities and are frequent, compared to smaller categories that occur in smaller quantities and are almost rare.

Table 1 Overview of attacks present in the dataset

S.No	Attacks	Descriptions
1	DDos	A DDoS attack involves flooding systems with massive traffic from different systems simultaneously, with the aim of denying service to authorized users.
2	Dos	This attack is very similar to DDoS, but the difference is that it comes from a single source with the goal of flooding the system and disrupting service.
3	Miri	A notable DDoS attack uses IoT devices as a botnet to launch large-scale attacks, highlighting vulnerabilities in IoT systems.
4	Recon	This attack relies on scanning systems to gather information in order to discover vulnerabilities that can be exploited.
5	Spoofing	In this type of attack, the attackers deceive systems by spoofing identities or IP addresses in order to gain authorized access.
6	Web	this attack target web applications and infrastructure, exploiting vulnerabilities to breach and manipulate digital platforms.
7	Brute Force	In this attack, attackers use all possible methods to crack passwords or usernames, and this attack is usually carried out using automated tools.

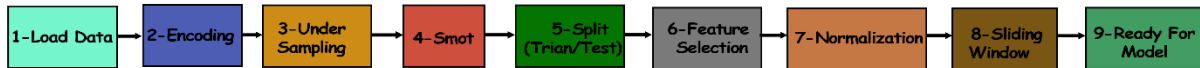
Table 2 Dataset description.

S.No	Attacks	Value
1	Benign	658,218 Record
2	DDos	3,588,628 Record
3	Dos	968,870 Record
4	Miri	316,100 Record
5	Recon	38,029 Record
6	Spoofing	21,509 Record
7	Web	1,398 Record
8	Brute Force	2,260 Record
Total		5,873,083 Record

3.2 Data Preprocessing

We performed several operations on the CIC-IoT-2023 [7] dataset to address its existing problems and prepare it for training with the appropriate model, as shown in Figure 1.

Figure 1



3.2.1 Load Data

The dataset was available to everyone in CSV files in pcap format, and we used pandas to load this data and remove duplicate values to make it ready for the next step.

3.2.2 Encoding

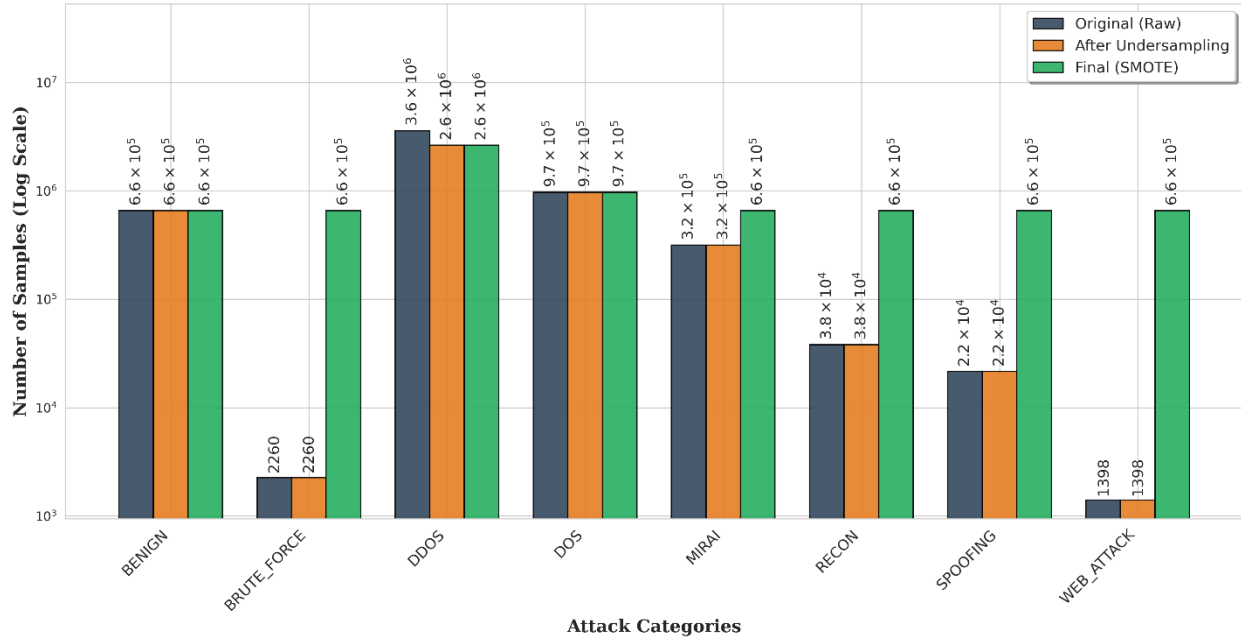
Working with neural networks requires working with numbers. Labels don't necessarily mean numbers, so all existing labels must be converted to numbers. We did this using the LabelEncoder technology from the Scikit-Learn [27] library to ensure that all existing labels were converted to symbolic numbers to complete the learning process.

3.2.3 Under-Sampling

The CIC-IoT-2023 [7] dataset suffers from severe imbalance and bias towards larger classes, making it impossible for the model to accurately detect smaller classes after training. To solve this problem, we implemented a hybrid method using RandomUnderSampler [8] to reduce the size of large classes and SMOT [9] to boost smaller classes, Figure 2 illustrates the distribution of class sizes for Under-Sampling, SMOT [9], and Original Samples.

Figure 2

CIC-IoT-2023 Class Distribution: Balancing Impact



3.2.4 SMOT [9]

To address the other data imbalance, which concerns the scarcity of smaller classes, we employed SMOT [9] (Selective Mobilization of Opportunities) technology. This technique increases the number of smaller classes by creating artificial samples similar to existing ones. Ultimately, this solves the problem of the scarcity of smaller samples, allowing the model to see a larger number of smaller classes. This, in turn, prevents bias towards larger classes, resulting in more stable training outcomes for all classes, Figure 2 shows the size of the classes after redistribution and balancing.

3.2.5 Data Splitting

We divided our dataset into two groups: Train and Test, at 80% and 20% respectively. The training group was further divided into a training group and a verification group during training, at 90% and 10% respectively.

3.2.6 Feature Selection

To improve model performance, accelerate the training process, and reduce computational complexity, we needed to decrease the number of classes in the dataset. CIC-IoT-2023 [7] contained 46 features, and a large number of these caused significant computational complexity and noise. As shown in the Lower Triangle Correlation Heatmap in Figure 3, there is a strong correlation between features such as Weight and Number, as well as high correlations between Max and Std. To address these issues, we selected only the 28 best features using the Random Forest Importance (RFI) [10] technique. We trained this algorithm on the training data to calculate the Mean Decrease in Impurity (MDI), then ranked the Features according to their importance, and finally selected the 28 best features. As shown in Figure 4, the final class order is as follows: IAT (Inter-Arrival Time) is the largest class the model relies on for class separation, contributing over 25% to the model's decision-making process. Other features, such as Magnitue and ack_count, also demonstrate significant importance in the model's ability to separate classes. To confirm the ability of the top 28 features to separate the classes, a visual representation was created using t-SNE [28]. In Figure 5, we find that the selected classes are highly effective in separating the classes. Specifically, the MIRAI, DDOS, and BENIGN traffic classes form well-defined and isolated clusters, confirming that the feature selection process was highly efficient.

Figure 3

Feature Inter-Correlation Heatmap (Top 25)

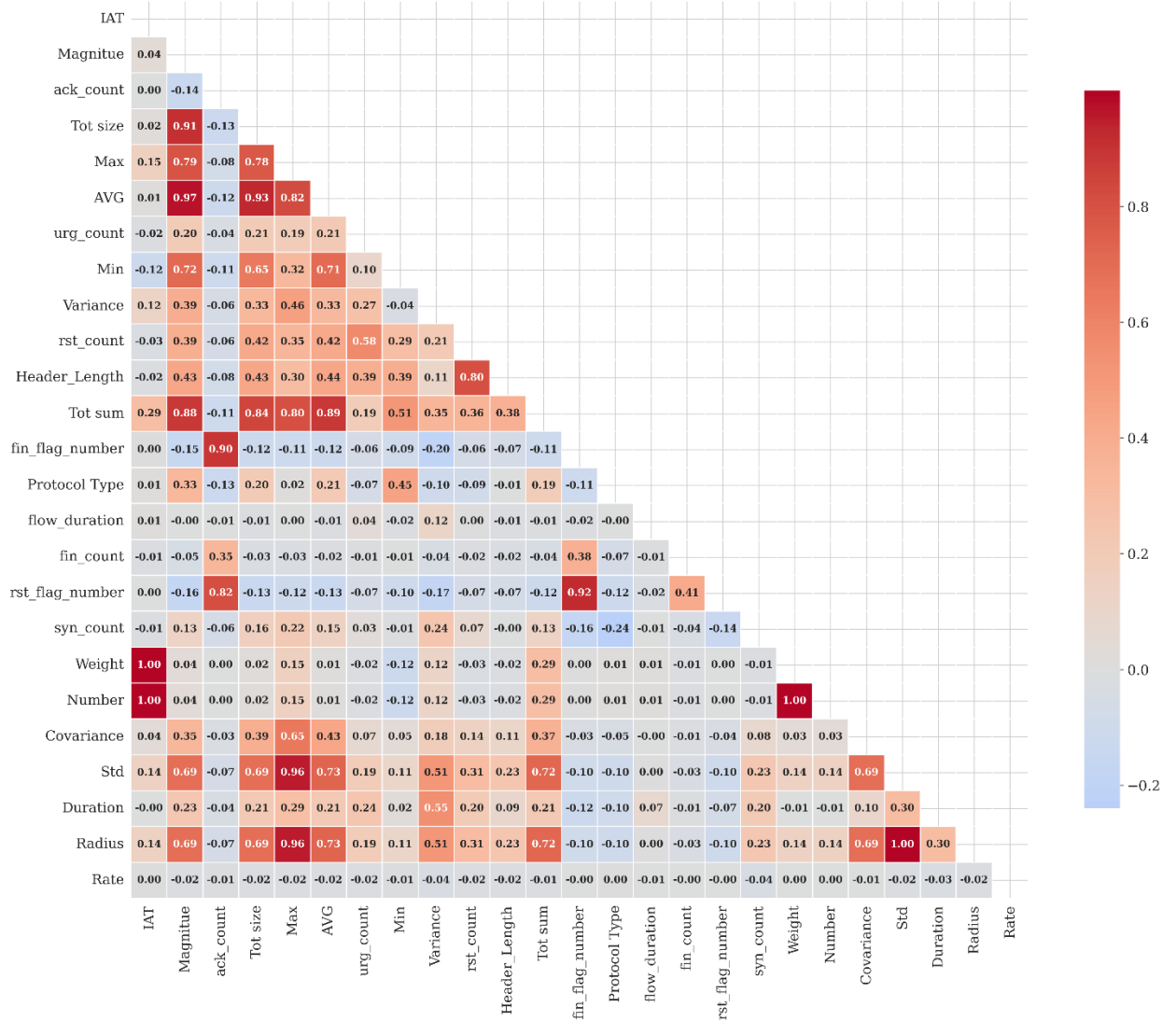


Figure 4

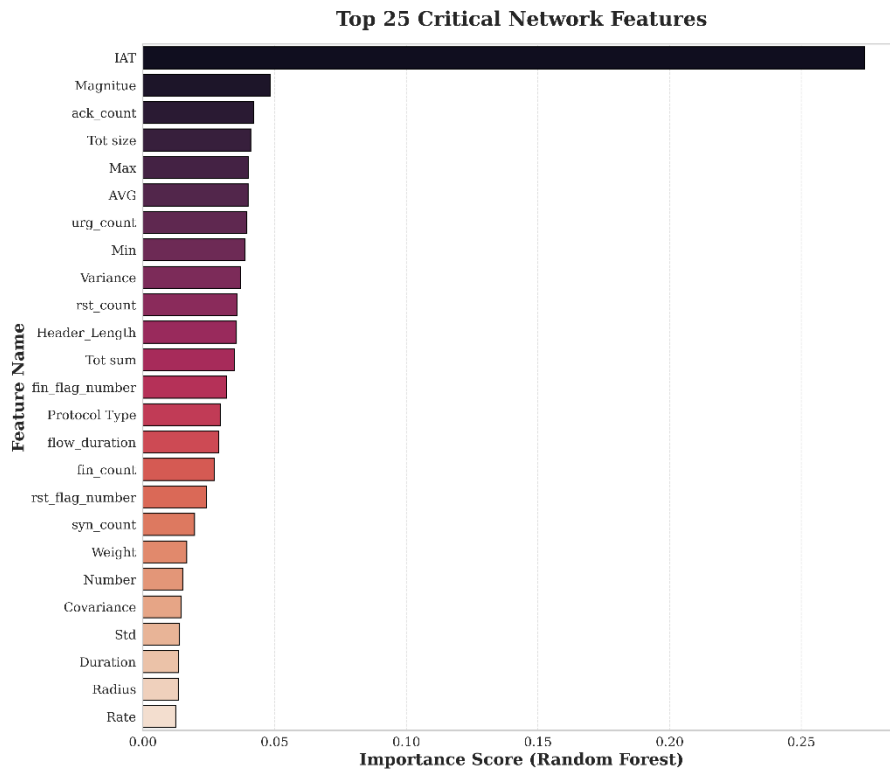
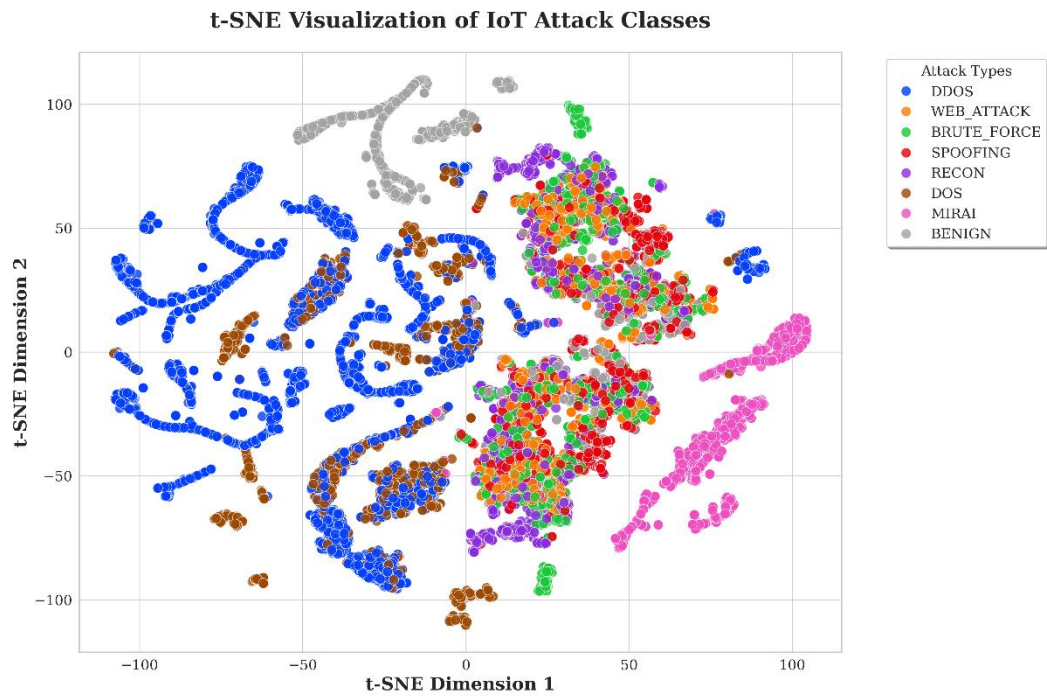


Figure 5



3.2.7 Data Normalization

To handle standard deviations and different means within the data, we used StandardScalar from the Scikit-learn [27] library, which scales the input data, resulting in a mean of zero standard deviation.

3.2.8 Sliding Window [1]

To leverage the temporal characteristics inherent in network traffic flows, a Sliding Window [1] technique was implemented to transform the dataset from a traditional two-dimensional (2D) format into a three-dimensional (3D) temporal structure. This process groups consecutive records into fixed-length sequences, enabling the deep learning model to learn the dependencies between historical and current network events.

Equation 1: Window Representation

Each input sequence at s_t at time t is constructed by aggregating ten consecutive records. Given a window size $T = 10$ and a feature dimension $d = 28$, the sequence is mathematically represented as follows:

$$S_t = [x_t, x_{t+1}, \dots, x_{t+T-1}] \in \mathbb{R}^{T \times d} \quad (Eq. 1)$$

For the classification task, a "last-record labeling" approach was adopted. Each generated sequence is mapped to the label corresponding to the final record within that window, This ensures that the model's prediction reflects the most recent state of the network, informed by its immediate temporal context.

Equation 2: Total Generated Samples

Using a sliding window [1] with a stride of one $S = 1$ the total number of generated sequences $N_{sliding}$

Is slightly less than the original number of records. The new dataset size is calculated using the relationship shown in Equation (2):

$$N_{\{sliding\}} = N_{\{total\}} - T + 1 \quad (Eq. 2)$$

Where N_{total} represents the total number of records in the unified dataset after the preprocessing stages. This transformation preserves data continuity while providing the necessary temporal depth required for training the proposed hybrid architecture.

3.3 Model Architecture

This section presents a comparison of the five architectures for each model, illustrating the architectural form of each model and the description of each layer. We also present the hyperparameters used in the training process, as shown in Table 3.

3.3.1 AE-LSTM-CNN [2]

This architecture uses a hybrid model consisting of several AE, LSTM, and CNN models, leveraging the unique characteristics of each model and then combining them. The Autoencoder's function is to extract spatial features and compress dataset features to reduce noise while preserving intrinsic characteristics. The second component, the Long Short-Term Memory (LSTM), is responsible for detecting time patterns identified by the Sliding Window. This component works in parallel with the Autoencoder. The third component in this architecture is the Convolutional Neural Network (CNN), which takes the output from the first two components and performs feature purification and final classification of the model, thus increasing the accuracy of the final classification. As shown in Figure 6, In algorithm number 1, we explain the pseudo-code for the model structure, where we find that the model consists of several layers.

- Input Layer & Data Preparation

This layer receives the 3D array (*Batch, Window, Features*). The science behind using Windowing is to transform network data from mere discrete records into a "Temporal Context," allowing the model to see the attack as a sequence of events rather than a single isolated event.

- Branch 1: Autoencoder (AE)

This path acts as a spatial feature filter, containing a Flatten layer that temporarily breaks the time structure to analyze the overall relationships between all features in a single window. It also includes an Encoder layer that compresses features to force the model to ignore feature noise. Additionally, this path contains a Decoder layer that reconstructs the original features, acting as a filter and teaching the model to detect anomalies later. Finally, there is a Batch Normalization layer that stabilizes the training and prevents the Vanishing Gradient problem.

- Branch 2: LSTM (Long Short-Term Memory)

This branch focuses on discovering patterns and time relationships. 23 units were used to extract recurring patterns, and then `return_sequences=True` was used to force the model to retain the time dimension to enter the next CNN layer, thus preventing any data loss.

- Concatenation Layer (Fusion)

Here at this stage, we combine the features filtered from the first branch (AE) and the second branch (LSTM). This stage enables the model to have dual spatial and temporal views of features and is called Feature-Level Fusion.

- CNN Layer (Spatial Feature Extraction)

This layer, after merging, acts as a final filter aimed at revealing spatial features between features and detecting clusters of features that indicate the presence of an attack.

- Classification Stage

Here, we used the Dense layer, which acts as a higher-order thinking layer that links features together. We also used the LeakyReLU layer, which prevents cell death by allowing very small signals to pass through. In the final layer, we used Softmax, which converts the output into probabilities and then selects the class with the highest attack to be the final result.

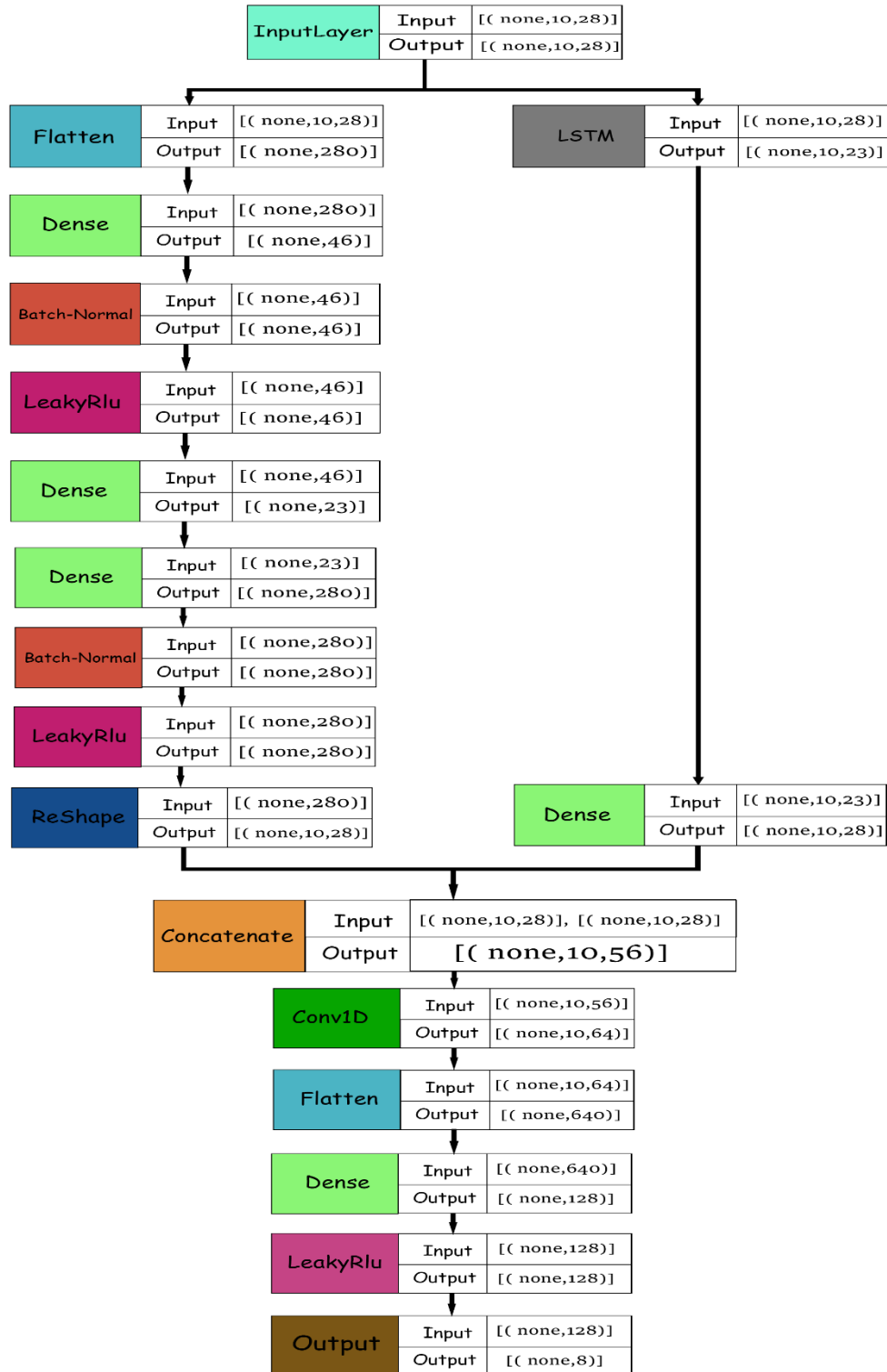


Figure 6 AE-LSTM-CNN

Algorithm 1: Hybrid AE-LSTM + CNN [2] Architecture

Input: *window_size, n_features, num_classes*

Output: Compiled Hybrid Model

1. Input Stage:

- Define input layer with shape: $(window_size, n_features)$
- Calculate $total_elements = window_size \times n_features$

2. Branch 1: Autoencoder (Static Feature Refinement)

- **Step 1.1:** Flatten input to 1D vector.
- **Step 1.2 (Encoder):** Pass through Dense(46) -> BatchNormalization -> LeakyReLU.
- **Step 1.3 (Decoder):** Pass through Dense(23) -> Dense($total_elements$).
- **Step 1.4:** Apply BatchNormalization -> LeakyReLU.
- **Step 1.5:** Reshape output back to $(window_size, n_features)$.

3. Branch 2: LSTM (Temporal Feature Extraction)

- **Step 2.1:** Apply LSTM(23) with $return_sequences = True$.
- **Step 2.2:** Pass through Dense($n_features$) to align dimensions.

4. Branch Integration (Concatenation):

- Merge outputs from Branch 1 and Branch 2 using Concatenate layer ($axis = -1$).

5. Classification Stage (CNN & Output):

- **Step 5.1:** Apply Conv1D (64 filters, kernel size 3, padding 'same').
- **Step 5.2:** Flatten the CNN output.
- **Step 5.3:** Pass through Dense(128) -> LeakyReLU.
- **Step 5.4:** Final Dense layer with $num_classes$ and Softmax activation.

6. Model Compilation:

- Use Adam Optimizer (Learning Rate = 0.001).
- Use Sparse Categorical Crossentropy loss function.

3.3.2 CAE-TCN [3]

This model employs a hybrid intrusion detection technique, combining a convolutional autoencoder (CAE) and a time convolutional network (TCN). CAE acts as an autoencoder, compressing data to reduce noise for the model and then decompressing it. This section focuses on spatial and local data detection. The model utilizes convolution and clustering techniques to reduce computational complexity. In another part of this architecture, Temporal Convolutional Networks (TCN) are used to detect time patterns. This section relies on strict time convolution, expanding the temporal perception range and enabling the model to capture long-term time dependencies without significant computational cost. Algorithm 2 details the steps for generating this architecture and integrating it with Sliding-Window, revealing that the architecture consists of several layers As shown in Figure 7.

- **Input Layer**

This layer receives data as time series (*window_size*, *n_features*) so that we can later extract spatial and temporal features from them.

- **Time Convolutional Network Path (TCN)**

This branch is designed for time path extraction. The residual block consists of two extended convolutions. We used *dilation_rate* to increase the receiving field without increasing the number of parameters. We used Shortcut Connection to adjust the number of channels to match the number of inputs with the outputs. We used Dropout and *BatchNormalization* layers to ensure model stability and prevent overfitting.

- **Attention Layer**

This layer calculates the weights for each time moment using tanh and softmax, helping the model focus on specific signals in the window that contain the attack.

- **Convolutional Auto-Encryptor Path (CAE)**

This path is dedicated to spatial feature discovery, where the Conv1D layer relies on graph compression and dimensionality reduction while preserving the spatial structure of the data. Similarly, the GlobalAveragePooling1D layer is used to extract hierarchical features and reduce computational complexity.

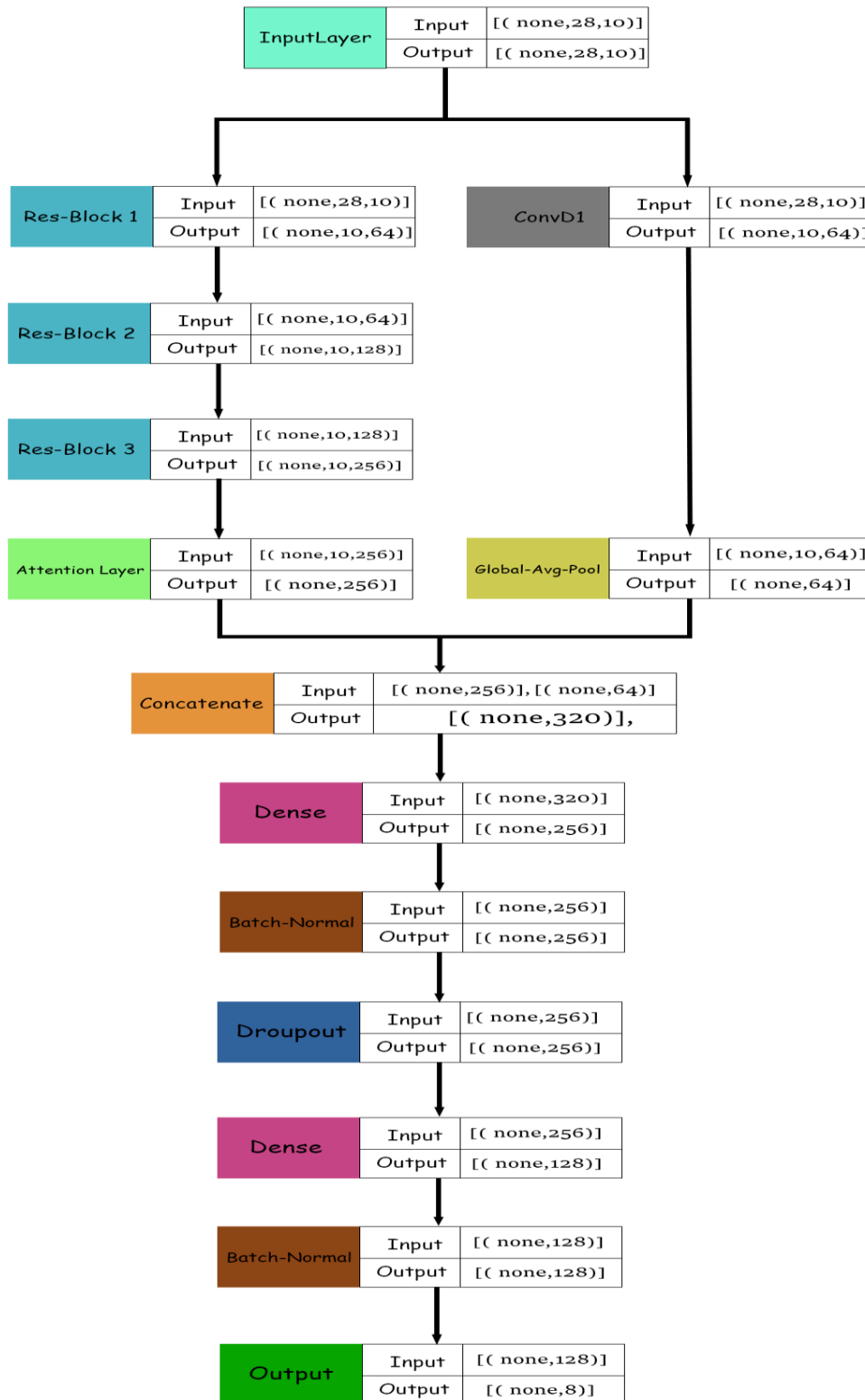
- **Feature Fusion**

After extracting the temporal and spatial features, this step involves combining the two branches, TCN and CAE, to form a "CAE-TCN module" using Concatenate(), enabling us to obtain a comprehensive representation of the features.

- **Dense Layers & Softmax Classification**

These layers act as common layers for the final representation of the data, and the final classification layer relies on the *softmax* classification function to complete the classification process.

Figure 7 CAE-TCN



Algorithm 1: CAE-TCN [3]

```
Input: window_size, n_features, n_classes
Output: Compiled Keras Model
# Define Attention Mechanism Layer
1: Class AttentionLayer:
2:     Initialize weights  $W$  with shape (input_dim, 1) and bias  $b$ 
3:     Function Call(input  $x$ ):
4:          $e = \tanh(x.W + b)$ 
5:          $a = \text{Softmax}(e)$  along time axis
6:         Return  $\sum(x.a)$  along time axis
# Define Residual Block (TCN Unit)
7: Function ResidualBlock( $x$ , filters, kernel, dilation):
8:     shortcut =  $x$ 
9:     If shortcut filters  $\neq$  target filters: shortcut =
Conv1D(1x1)(shortcut)
10:     $x = \text{Conv1D}(\text{filters}, \text{kernel}, \text{dilation})(x) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow$ 
Dropout(0.2)
11:     $x = \text{Conv1D}(\text{filters}, \text{kernel}, \text{dilation})(x) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow$ 
Dropout(0.2)
12:    Return ReLU( $x + \text{shortcut}$ )
# Main Model Architectures
13: InputLayer: visible = Input(shape=(window_size, n_features))
14: # TCN Path (Temporal Features)
15: tcn = ResidualBlock(visible, filters=64, dilation=1)
16: tcn = ResidualBlock(tcn, filters=128, dilation=2)
17: tcn = ResidualBlock(tcn, filters=256, dilation=4)
18: attn_out = AttentionLayer(tcn)
19: # CAE Path (Global Spatial Features)
20: cae = Conv1D(64, kernel=1, activation='relu')(visible)
21: cae = GlobalAveragePooling1D()(cae)
22: # Fusion and Classification
23: merged = Concatenate()([attn_out, cae])
24:  $x = \text{Dense}(256, \text{activation}='relu')(merged) \rightarrow \text{BatchNorm} \rightarrow \text{Dropout}(0.4)$ 
25:  $x = \text{Dense}(128, \text{activation}='relu')(x) \rightarrow \text{BatchNorm}$ 
26: outputs = Dense(n_classes, activation='softmax')(x)
# Model Compilation
27: model = Model(inputs=visible, outputs=outputs)
28: optimizer = Adam(learning_rate=0.001)
29: Compile model with "sparse_categorical_crossentropy" and "accuracy"
metric
30: Return model
```

3.3.3 CNN-LSTM [4]

This architecture relies on a hybrid method that combines CNN and LSTM to produce this CNN-LSTM [4] model. Its importance lies in extracting spatial and temporal features. CNN is used for spatial feature extraction due to its high capacity, and the output is then sent to the LSTM layer because of its superior ability to detect temporal patterns. The algorithm 3 outlines three layers of this detailed architecture, which aligns with the sliding-window principle. The architecture consists of several steps As shown in Figure 8.

- **Convolutional Neural Network (CNN) Layer**

This layer is used as a spatial feature extractor. It begins by applying a set of filters, multiplying the number of filters by their weights, and then summing the result into a single numerical value. The results are then passed through the *Relu* function to handle negative results. Following the Convolution layer comes the Poling layer, which reduces the dimensions of the array, thus helping to prevent overfitting.

- **Batch Normalization (BN) Layer**

The purpose of this layer is to address the problem of internal variance change that occurs as a result of changes in the input distribution. This layer is located between the CNN and LSTM layers, and it also accelerates the optimization process and reduces generalization errors by standardizing the data scale.

- **Long Short-Term Memory (LSTM) Layer**

This layer aims to extract temporal features after the spatial feature extraction process from the CNN. LSTM is distinguished by its ability to translate and store data over time using Memory Cells.

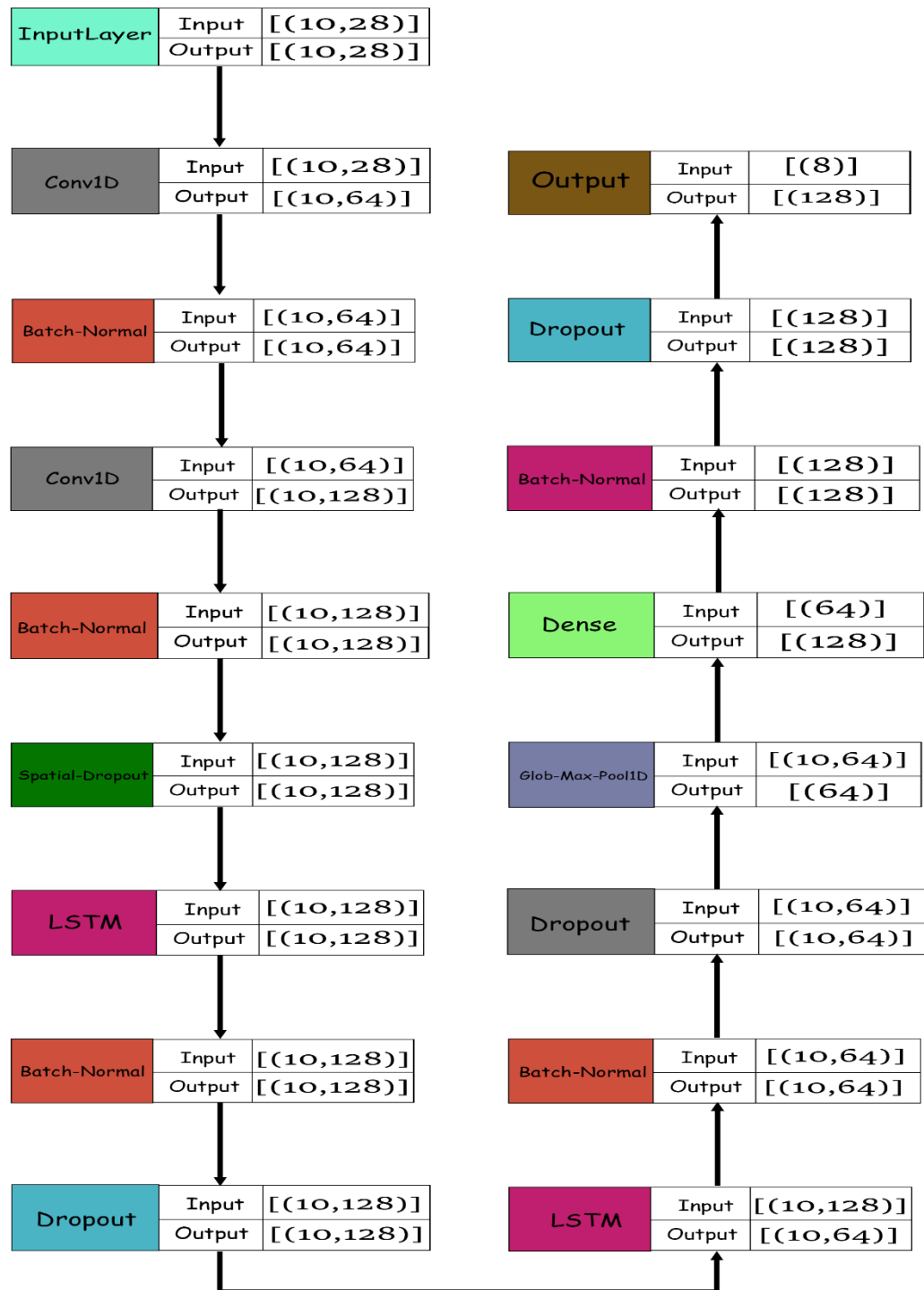


Figure 8 CNN-LSTM

Algorithm 3: CNN-LSTM [4]

```
Input: window_size, n_features, n_classes
Output: Trained Intrusion Detection Model
1. Initialization:
Define Sequential Model pipeline.
Input Shape: (window_size, n_features).
2. Stage 1: Shallow Spatial Feature Extraction:
Step 1.1: Apply Conv1D (64 filters, kernel_size=3, padding='same',
activation='relu').
Step 1.2: Apply BatchNormalization to stabilize feature maps.
Note: No pooling is used here to preserve temporal length for LSTM.
3. Stage 2: Deep Feature Learning:
Step 2.1: Apply Conv1D (128 filters, kernel_size=3, padding='same',
activation='relu').
Step 2.2: Apply BatchNormalization.
Step 2.3: Apply SpatialDropout1D (0.3) to drop entire feature maps
for regularization.
4. Stage 3: High-Dimensional Temporal Learning:
Step 3.1: Apply LSTM (128 units) with return_sequences = True.
Step 3.2: Apply BatchNormalization.
Step 3.3: Apply Dropout (0.2).
5. Stage 4: Feature Aggregation & Reduction:
Step 4.1: Apply LSTM (64 units) with return_sequences = True.
Step 4.2: Apply BatchNormalization → Dropout (0.2).
Step 4.3 (Critical): Apply GlobalMaxPooling1D to extract the most
salient features from the time sequence.
6. Stage 5: Classification Head:
Step 5.1: Fully Connected Dense (128 units, activation='relu').
Step 5.2: Apply BatchNormalization → Dropout (0.2).
Step 5.3: Final Dense layer (n_classes, activation='softmax').
7. Model Compilation:
Optimizer: Adam (Learning Rate = 0.001).
Loss Function: Sparse Categorical Crossentropy.
```

3.3.4 LSTM [11]

This architecture relies on the LSTM [11] model to capture crucial time patterns in intrusion detection. The model's intrusion contains memory cells that store features over time, allowing for the detection of hidden time patterns. In Algorithm 4, we illustrate the model's various layers. This model consists of several layers As shown in Figure 9 and handles sliding windows with a 10-packet size.

- **Deep Stacked LSTM Structure**

The model is designed with a cumulative architecture comprising 4 *LSTM* layers, where the number of units ranges from 256 to 128 to 64 to 32. The first layers extract general patterns, while deeper layers extract the intrinsic features that characterize each attack. Return Sequences are enabled for the first 3 layers to ensure the complete chronology is passed to the next layer, allowing the network to build a cumulative understanding of the event.

- **Average Pooling 1D (Feature Smoothing)**

This *AveragePooling1D* layer aims to reduce the existing light, allowing the model to focus more on average behavior rather than deviations. It also reduces the computational load by half-reducing the time dimensions.

- **Batch Normalization & LeakyReLU**

The *BatchNormalization* layer solves the internal distribution problem, allowing the model to learn stably and quickly, while *LeakyReLU* solves the Vanishing Gradient problem.

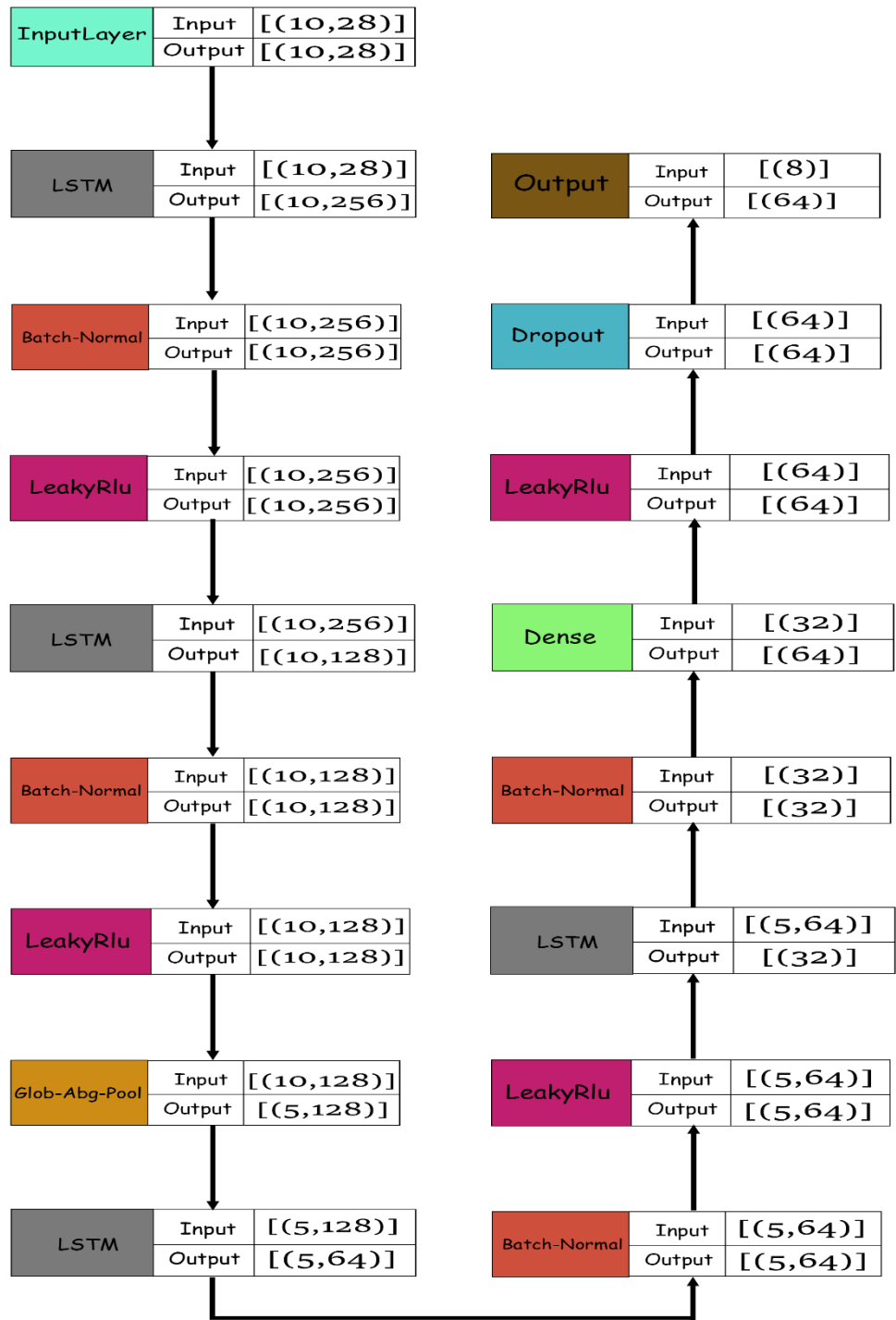
- **Final LSTM**

This layer is considered the decision-making layer, where it transforms the time series (matrix) into a single feature vector that represents a "summary" of the entire time window, in preparation for classifying it.

- **Dense & Dropout (Classification)**

This dense layer binds the extracted features, while the *Droupout* layer deletes 30% of the cells to prevent overfitting.

Figure 9 LSTM



Algorithm 4: LSTM [11]

```
Input: window_size, n_features, n_classes
Output: Optimized Intrusion Detection Model
1. Input Initialization:
Define Input layer: shape = (window_size, n_features).
2. Phase 1: High-Dimensional Temporal Extraction:
Step 1.1: Apply LSTM (256 units, return_sequences = True).
Step 1.2: Apply BatchNormalization → LeakyReLU (0.01).
Note: Captures broad and complex temporal patterns.
3. Phase 2: Feature Refinement and Downsampling:
Step 2.1: Apply LSTM (128 units, return_sequences = True).
Step 2.2: Apply BatchNormalization → LeakyReLU (0.01).
Step 2.3: Apply AveragePooling1D (pool_size = 2).
Note: Reduces dimensionality and noise while retaining trend
information.
4. Phase 3: Intermediate Temporal Learning:
Step 3.1: Apply LSTM (64 units, return_sequences = True).
Step 3.2: Apply BatchNormalization → LeakyReLU (0.01).
5. Phase 4: Sequence Compression (Bottleneck):
Step 4.1: Apply LSTM (32 units, return_sequences = False).
Note: Collapses the temporal dimension into a fixed feature vector.
Step 4.2: Apply BatchNormalization.
6. Classification Head :
Step 5.1: Fully Connected Dense (64 units).
Step 5.2: Apply LeakyReLU (0.01) → Dropout (0.3).
Step 5.3: Output Layer: Dense (n_classes, activation='softmax').
7. Model Compilation:
Optimizer: Adam (Learning Rate = 0.001).
Loss: Sparse Categorical Crossentropy.
```

3.3.5 CNN-BILSTM [6]

This architecture is based on a hybrid architecture composed of CNN and BILSTM. CNN is used for its high capacity for extracting spatial features, while BILSTM is used for its ability to capture temporal dependencies in forward and backward directions. This dual capability is crucial for detecting temporal patterns between attacks. In Algorithm 5, we illustrate the layered structure of this hybrid architecture, showing that the model consists of several layers As shown in Figure 10.

- **Input Reshaping (Alignment)**

This layer reshapes the data to conform to the Conv1D format, where the data is converted as a vertical vector ready for subsequent operations.

- **Convolutional Layer (Spatial Learning)**

This CNN layer is based on discovering spatial features and initial relationships between network features. This layer relies on local connectivity through weight sharing, which reduces computational effort. We used *LeakyReLU* in this layer to ensure training speed and stability.

- **Batch Normalization**

This layer maintains training stability by reducing the Moodle's reliance on draw out and preventing overfitting. Its greatest importance lies in addressing the covariate shift that can occur when the value distributions from each layer constantly change because the weights of the preceding layers are updated.

- **Bidirectional LSTM (Bi-LSTM) - "The Dual Perspective"**

This part is considered the most important and distinctive of the architecture, as, unlike regular LSTM, BILSTM uses two-way layers to understand the time context from both the back and front directions. This is achieved by enabling two LSTM layers, one working on the back view of contexts and the other working on understanding the frontal context, which allows it to understand the context in a more comprehensive and broader way and gives it the ability to understand the evolution that may occur in attacks such as DDoS attacks.

- **Global Average Pooling**

This layer calculates the average for each feature map over time. This significantly reduces the number of parameters, protecting the model from overfitting and making it more efficient at extracting global features.

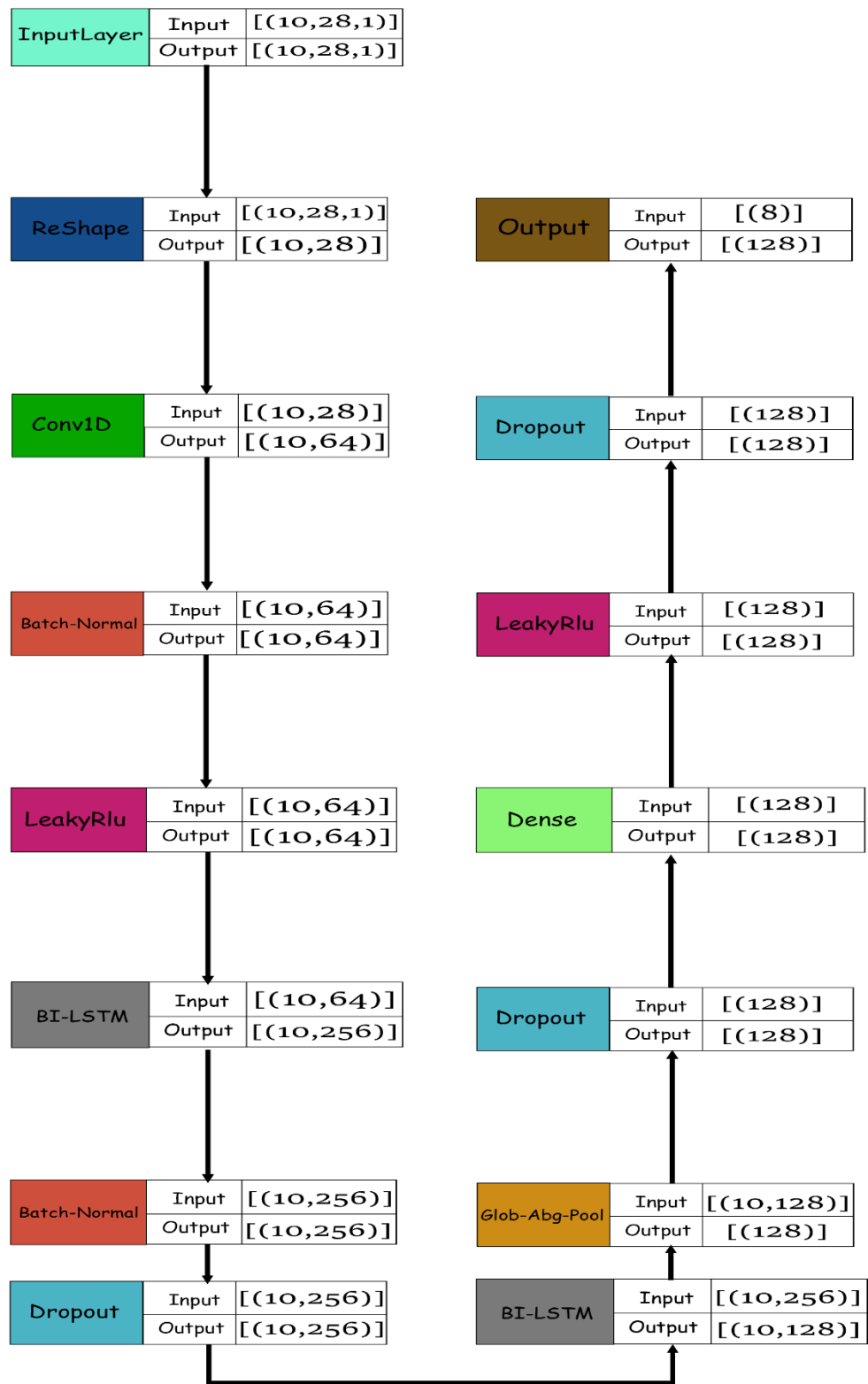
- **L2 Regularization & Dense Layers**

We added this *kernel_regularizer = l2(0.001)* to the dense layer to force the model to learn more general patterns by adding a penalty proportional to the square of the weights, thus preventing excessive weight growth, maintaining stability, and improving classification accuracy.

- **Softmax Classification**

This layer, through the *softmax* function, converts network outputs into probabilities and then selects the appropriate attack type based on the highest probability.

Figure 10 CNN-BiLSTM



Algorithm 5: CNN-BiLSTM [6]

Input: *window_size, n_features, num_classes*
Output: Classified Attack Category
Input & Reshaping Stage:
Define Input layer: (*window_size, n_features, 1*).
Apply Reshape to (*window_size, n_features*) to ensure mathematical compatibility with 1D convolution.
Spatial Feature Extraction (CNN):
Apply Conv1D (64 filters, kernel=3) to extract local spatial features.
Apply BatchNormalization and LeakyReLU (0.01) to stabilize gradients.
Stacked Bidirectional Temporal Learning (Bi-LSTM):
Layer 1: Bidirectional LSTM (128 units) with *return_sequences = True*. Captures full context by processing sequences in forward and backward directions.
Layer 2: Bidirectional LSTM (64 units) with *return_sequences = True*. Refines temporal patterns and prepares the sequence for global aggregation.
Feature Fusion & Aggregation:
Apply GlobalAveragePooling1D to condense the temporal information into a global feature vector.
Apply Dropout (0.3) to prevent overfitting.
Deep Classification Head:
Dense Layer: 128 units with L2 Regularization (*lambda = 0.001*).
Apply LeakyReLU and Dropout (0.4).
Output Stage:
Dense Layer: *num_classes* with Softmax (for multi-class) or Sigmoid (for binary).

3.4 Training & Testing

Table 3 details all the significant parameters used in the training process, including the learning rate. We used *EarlyStopping* with a scheduler, which automatically reduces the learning rate when the model stops learning and begins overfitting.

Table 3 Hyperparameters used in this research

Hyperparameter	Value	Description
Learning rate	0.001	Controls the step size in weight updates during training
Batch size	256	Number of samples processed before updating the model parameters
Epochs	200	Total number of iterations over the training dataset
Loss function	Categorical Crossentropy	Loss function for multi-class classification tasks
Optimizer	Adam	Optimizer for model training using adaptive momentum estimation
Activation function	ReLU/Softmax	ReLU for hidden layers and Softmax for output layer
Patience (early stopping)	10	Number of epochs without improvement before stopping training
Window Size	10	The number of samples we send to Modle for learning at the same time

4.0 Results And Analysis

This section contains all the results and detailed final evaluation of the five models tested in this research of the proposed intrusion detection system, All parameters were fixed for all models, such as the Learning Rate, number of Epochs, data partitions, dataset size, testing environment, and everything else. The only change was in the model architecture to obtain accurate results that allow us to evaluate each model accurately and transparently.

4.1 Experimental environment

The experimental setup for this research was implemented using Kaggle, leveraging its high-performance cloud-based GPU for efficient model training and evaluation. The implementation utilized key Python libraries including Keras and TensorFlow for deep learning model development ,and NumPy and Pandas for numerical computations and data manipulation . The dataset was standardized using StandardScaler , and LabelEncoder to encode labels . A multi-model approach involving AE-LSTM-CNN [2], CAE-TCN [3] ,CNN-LSTM [4], LSTM [11], and CNN-BILSTM [6] was adopted for this study, where each model was developed using the Sequential API and incorporated Dropout and MaxPooling1D layers. Categorical cross-entropy was chosen as the loss function, optimized by the Adam algorithm to ensure efficient convergence. Furthermore, EarlyStopping was integrated into the workflow to maintain optimal generalization and avoid overfitting. The research evaluated these models using detailed classification reports and various graphical tools, such as calibration curves and confusion matrices, to provide a holistic view of their performance in detecting network intrusions.

4.2 Model Data Distribution

We used the CIC-IoT-2023 [7] dataset because it contains a vast collection of real-world data that simulates a real IoT environment, and because it includes approximately 33 different types of attacks, which we later divided into 7 main attacks as shown in Table 1.

4.3 Evaluation Metrics

We used several evaluation metrics in this research to measure the strengths and weaknesses of each model from multiple perspectives. The metrics we used were Accuracy, Precision, Recall, F1 Score, AUC Score, MCC Score, and Kappa Score.

The accuracy scale indicates the degree of convergence or agreement between the expected results and the actual values output by the model. The formula for this scale is expressed in the following equation:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \quad (1)$$

Precision represents the proportion of positive cases that were accurately identified compared to the total number of cases predicted to be positive. It is a key metric in model evaluation. Accuracy is calculated using the following equation:

$$\text{Precision} = \frac{TP}{TP+FP} \quad (2)$$

Recall quantifies the extent to which a model successfully detects all pertinent cases within a dataset. High recall values demonstrate the model's effectiveness in retrieving a larger proportion of true positives, signifying robust target class identification. It is determined by the following expression:

$$\text{Recall} = \frac{TP}{TP+FN} \quad (3)$$

The F1-score is a comprehensive performance metric that integrates both precision and recall to evaluate a model's classification accuracy. A higher F1-score reflects superior overall effectiveness in identifying positive samples, making it an essential tool for assessing model reliability. It is defined as the harmonic mean of precision and recall, as represented in the following formula:

$$\text{F1-Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

The area under the ROC curve (AUC) represents the probability that the model, if given a randomly chosen positive and negative example, will rank the positive higher than the negative, and it's represented in the following formula :

$$\text{AUC-Score} = \int_0^1 \text{TPR}(\text{FPR})d\text{FPR} \quad (5)$$

The Matthews Correlation Coefficient (MCC) was utilized to evaluate the model's performance across all categories. It provides a balanced assessment by incorporating the entire confusion matrix, making it superior for imbalanced datasets. The score ranges from -1 to +1, where +1 indicates perfect classification.

$$\text{MCC-Score} = \frac{TP*TN-FP*FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \quad (6)$$

Cohen's Kappa was used to evaluate classification consistency across the eight classes. By accounting for random chance agreement, it provides a more rigorous measure of reliability than standard accuracy, where a score of +1 indicates perfect agreement.

$$kappa = \frac{p_o - p_e}{1 - p_e} \quad (7)$$

4.4 Data Imbalance

We performed the training process using SMOT [9] to increase the size of the small classes in order to obtain balanced results for the small classes ,We have observed a significant improvement in the performance of the model for small-sized classes, SMOTE [9] is widely recognized as a robust and effective baseline method for oversampling in various domains.

We trained the five models using SMOT [9] to address data imbalances and the Sliding Window Approach to enable the model to detect time patterns in attacks. Since some attacks rely on repetition and hijacking, using a 10-packet size Sliding Window system allows the model to easily detect these patterns, especially when using a model with a cache such as LSTM, BILSTM[6], or TCN.

4.5 Analysis

4.5.1 AE-LSTM-CNN [2]

After training this model Sliding Window Approach, we obtained results of 98.44 % Accuracy, 97.80 % Precision, 97.79 % Recall, 0. 9995 AUC Score, and 0. 9810 MCC Score , . 9810 Kappa Score , We achieved high performance in the small classes of this model thanks to the use of SMOT [9] technology to address imbalances and the use of Sliding Window to explore time patterns within attacks. Figure 11 illustrates the Confusion Matrix of the model's training process, demonstrating high balance and accuracy for the model across all classes, particularly those with lower arguments such as Brute Force, Figure 12 also illustrates the learning curve and the model's behavior during the overall training process through two indicators: Training Accuracy and Validation Accuracy. The close alignment of these indicators suggests a stable training process free from overfitting, and Figures 13 and 14 show the Roc Curve and Precision Recall Curve of the model, Table 4 also shows the calculation for the Confusion Matrix, and we also shows the different performance metrics for each category in Table 5 also.

Figure 11

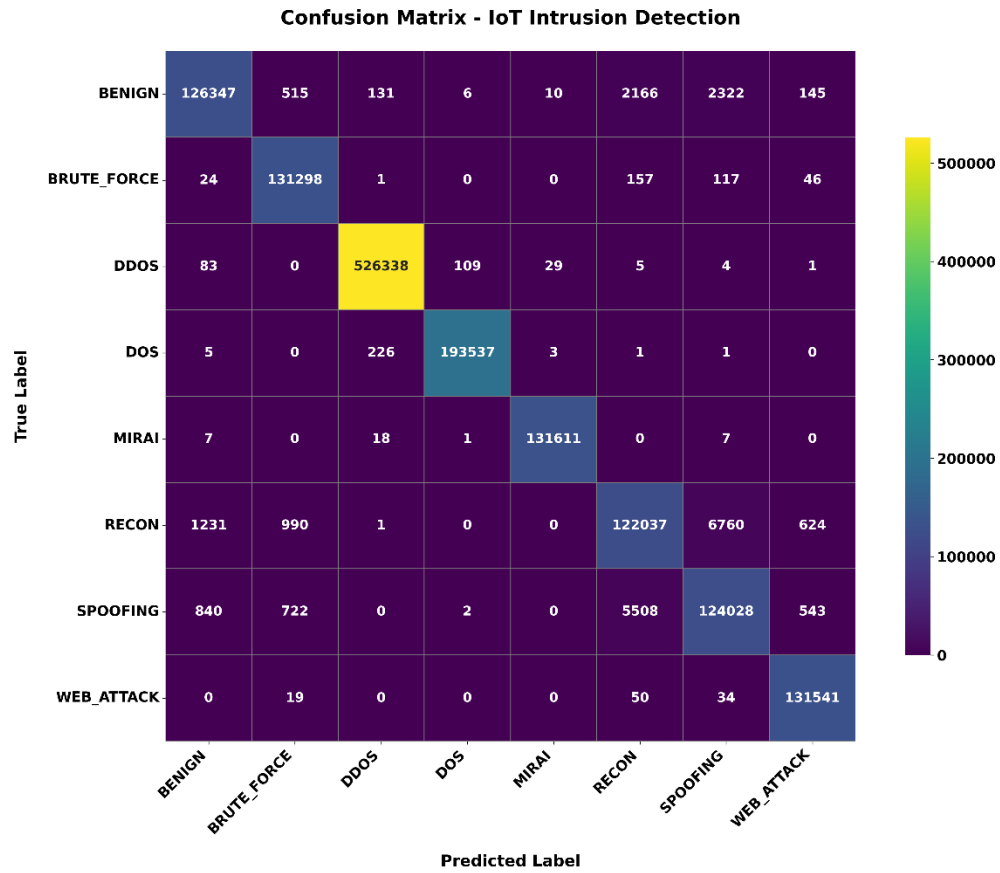


Figure 12

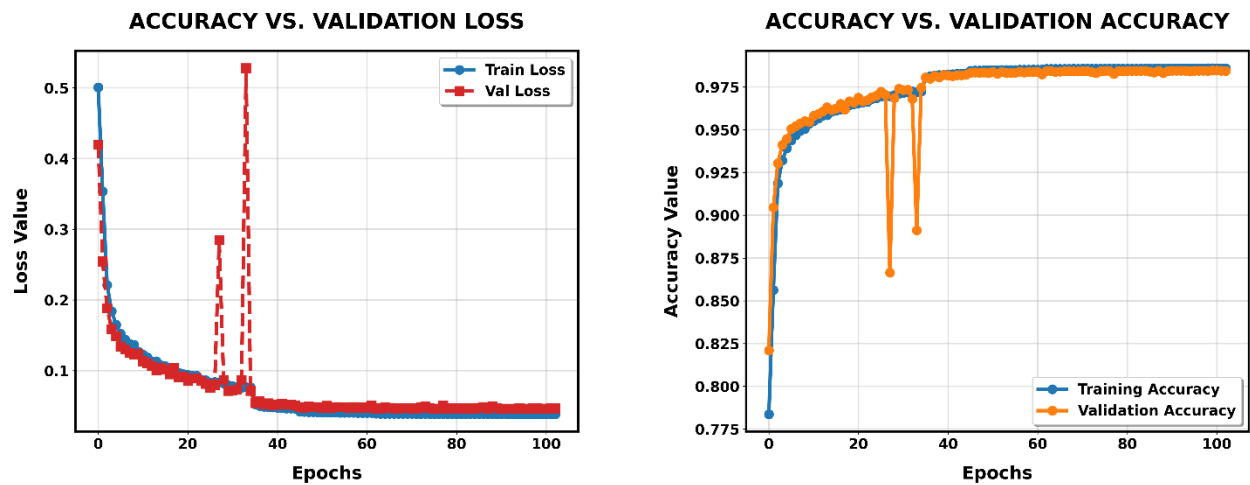


Figure 13

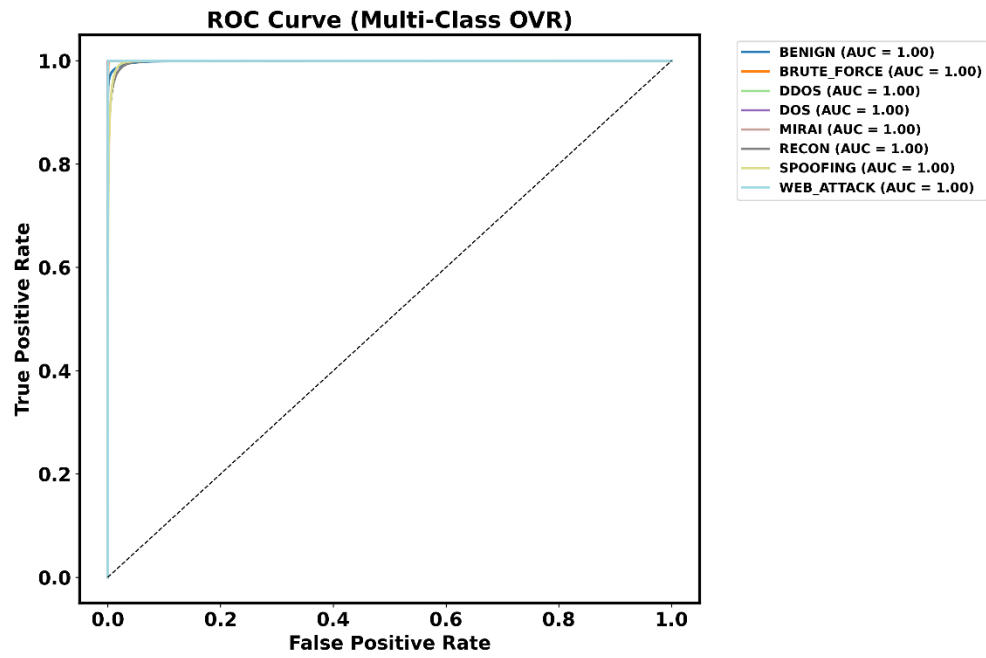


Figure 14

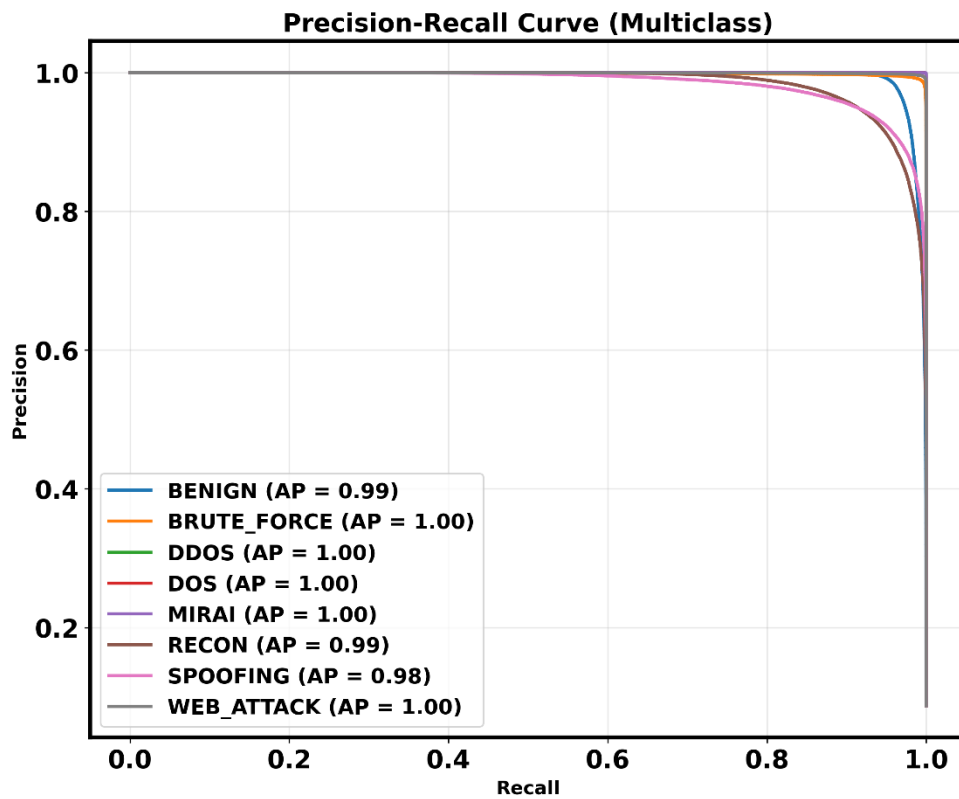


Table 4

Class	TP	FN	FP	TN
BENIGN	126347	5295	2190	1376369
BRUTE_FORCE	131298	345	2246	1376312
DDOS	526338	231	377	983255
DOS	193537	236	118	1316310
MIRAI	131611	33	42	1378515
RECON	122037	9606	7887	1370671
SPOOFING	124028	7615	9245	1369313
WEB_ATTACK	131541	103	1359	1377198

- True Positive (TP): Correctly predicted instances for the specific class.
- False Positive (FP): Incorrectly predicted as target class (Sum of Column - TP).
- False Negative (FN): Target class predicted as something else (Sum of Row - TP).
- True Negative (TN): Correctly predicted as NOT belonging to target class.

Table 5

Class	Acc(%)	Pr(%)	Rc(%)	F1(%)
BENIGN	99.50	98.30	95.98	97.12
BRUTE_FORCE	99.83	98.32	99.74	99.02
DDOS	99.96	99.93	99.96	99.94
DOS	99.98	99.94	99.88	99.91
MIRAI	100.00	99.97	99.97	99.97
RECON	98.84	93.93	92.70	93.31
SPOOFING	98.88	93.06	94.22	93.64
WEB_ATTACK	99.90	98.98	99.92	99.45

Our Observation about the proposed AE-LSTM-CNN [2] framework leverages the complementary strengths of deep learning architectures to establish a robust intrusion detection system. By utilizing LSTM for temporal analysis and CNN for spatial feature extraction, the model achieves remarkable improvements in accuracy, precision, recall, and F1 scores compared to traditional methods. This framework also has a very important feature, which is the speed of learning, thanks to the two branches, CNN and LSTM, which work in parallel, making the model learn exceptionally well and quickly.

4.5.2 CAE-TCN [3]

This model CAE-TCN [3] delivered outstanding performance and high speed in the training process thanks to its robust architecture, achieving Accuracy 98.56%, Precision 97.99%, Recall 97.96%, F1-Score 97.9%, AUC Score 99.9%, MCC Score 98.2%, and Kappa Score 98.2%. The model delivered high and consistent performance in small categories thanks to the time pattern detection feature within the TCN that describes complex attacks such as WEB_ATTACK, RECON, and BRUTE_FORCE. In Figure 15, we illustrate the model's Confusion Matrix and observe within it the balanced performance of the model for all its classes. The model performed fairly consistently in the training process, as shown in the learning curve diagram in Figure 16, which illustrates the convergence between two indicators: Train Accuracy and Validation Accuracy. Figures 17 and 18 also illustrate the ROC Curve and Precision Recall Curve diagrams. Table 6 shows the calculation for the Confusion Matrix, which illustrates the number of positive and negative predictions for each of the 8 dataset categories, also Table 7 also illustrates the different performance metrics for each category.

Figure 15

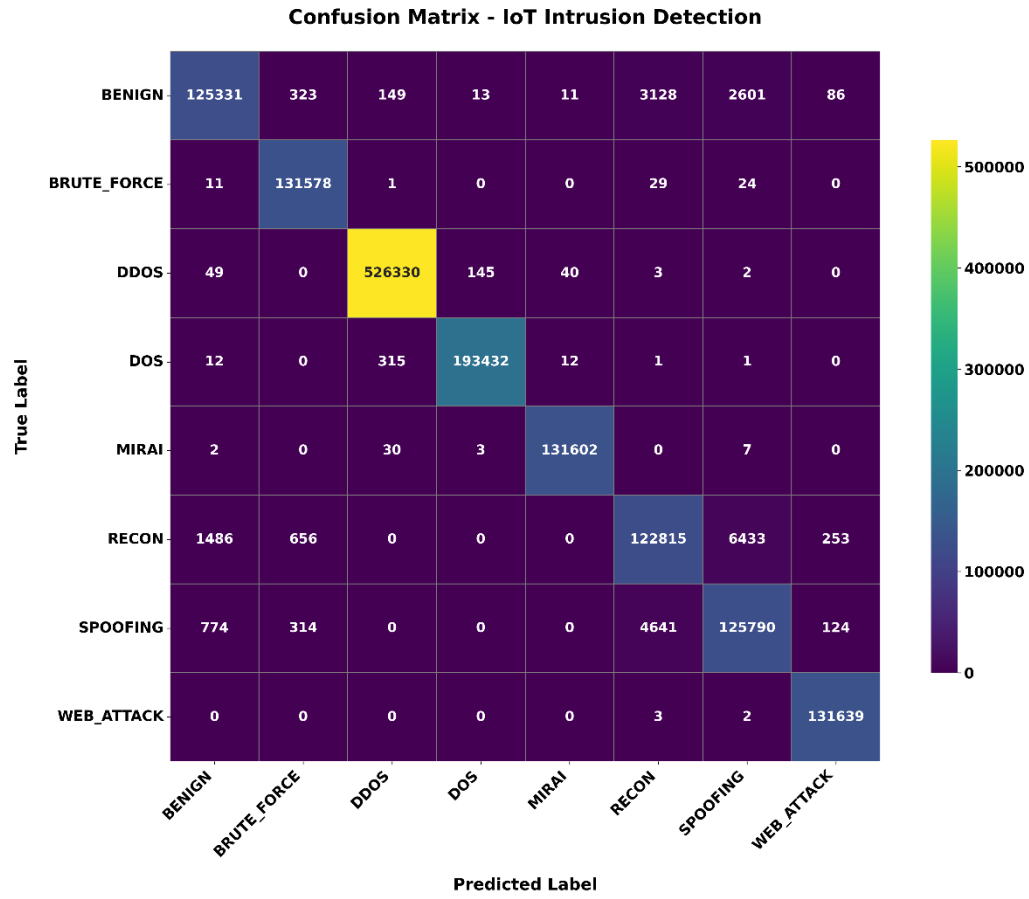


Figure 16

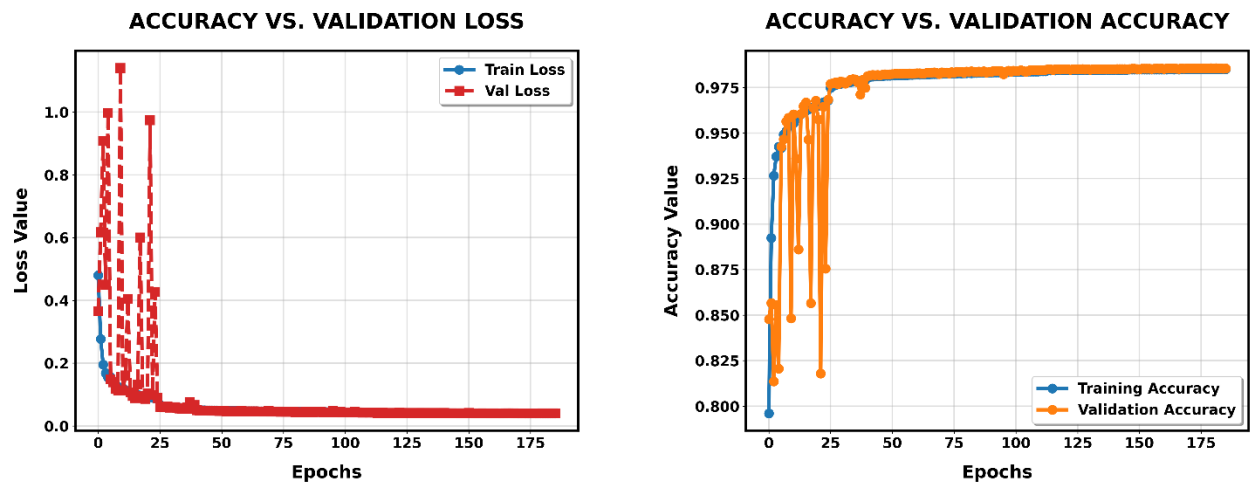


Figure 17

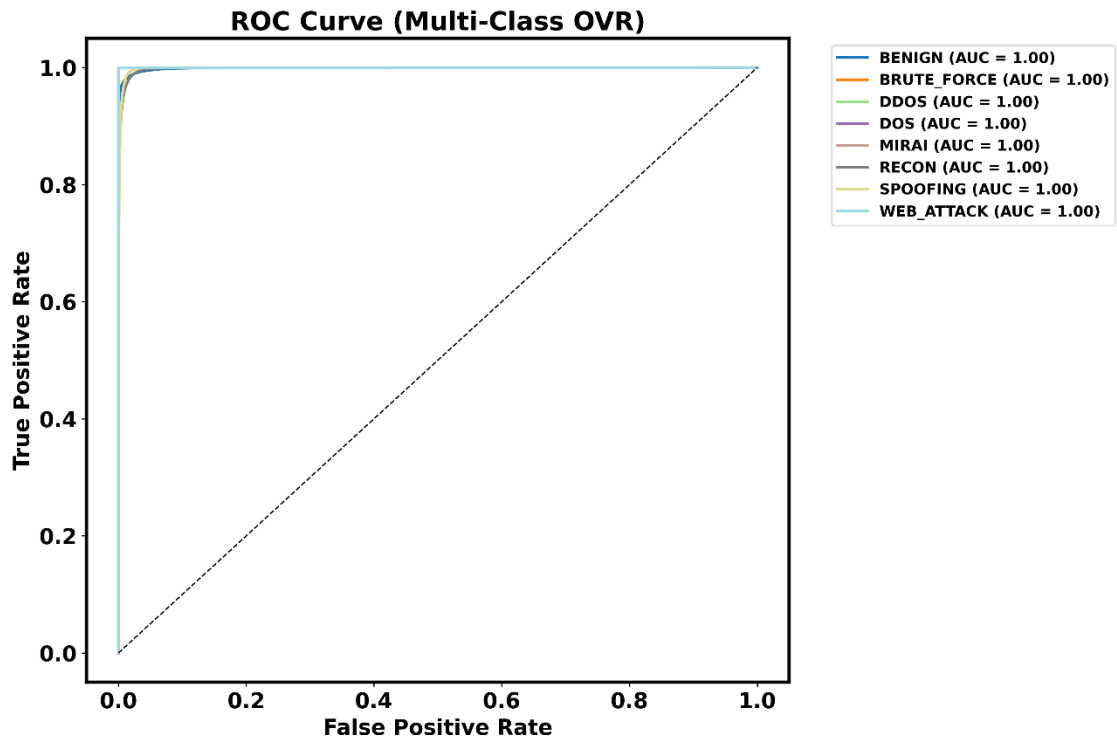


Figure 18

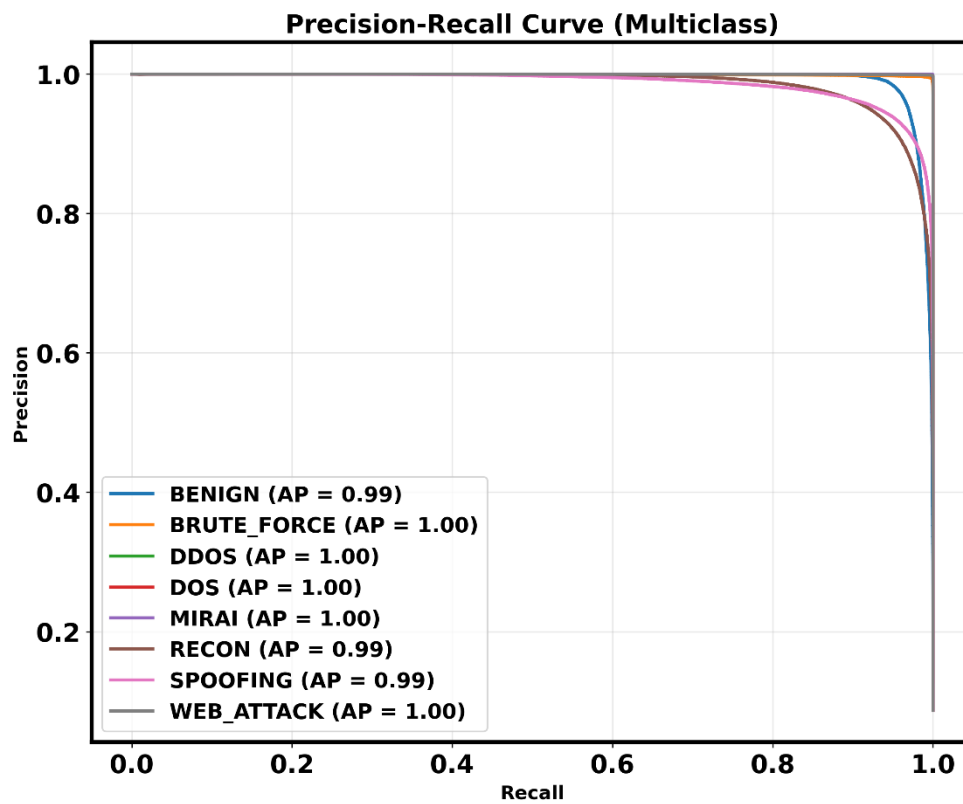


Table 6

Class	TP	FN	FP	TN
BENIGN	125331	6311	2334	1376225
BRUTE_FORCE	131578	65	1293	1377265
DDOS	526330	239	495	983137
DOS	193432	341	161	1316267
MIRAI	131602	42	63	1378494
RECON	122815	8828	7805	1370753
SPOOFING	125790	5853	9070	1369488
WEB_ATTACK	131639	5	463	1378094

- True Positive (TP): Correctly predicted instances for the specific class.
- False Positive (FP): Incorrectly predicted as target class (Sum of Column - TP).
- False Negative (FN): Target class predicted as something else (Sum of Row - TP).
- True Negative (TN): Correctly predicted as NOT belonging to target class.

Table 7

Class	Acc(%)	Pr(%)	Rc(%)	F1(%)
BENIGN	99.43	98.17	95.21	96.67
BRUTE_FORCE	99.91	99.03	99.95	99.49
DDOS	99.95	99.91	99.95	99.93
DOS	99.97	99.92	99.82	99.87
MIRAI	99.99	99.95	99.97	99.96
RECON	98.90	94.02	93.29	93.66
SPOOFING	99.01	93.27	95.55	94.40
WEB_ATTACK	99.97	99.65	100.00	99.82

4.5.3 CNN-LSTM [4]

This CNN-LSTM [4] model performed well, but it wasn't the highest-performing or fastest model in the training process, achieving Accuracy 97.02%, Precision 95.89%, Recall 95.82%, F1-Score 95.81%, AUC-Score 99.8%, MCC-Score 96.3%, and Kapp-Score 96.3%. Figure 19 shows the Moodle's Confusion Matrix, and we observe a noticeable increase in false alarms. Figure 20 shows the Moodle's Learning Curve, which demonstrates a degree of stability in the Moodle's learning process and a significant convergence in Training Accuracy and Validation Accuracy. Figures 21 and 22 also show the ROC-Curve and Precision-Recall-Curve graphs, Table 8 shows the calculations for the Confusion Matrix of the model, Table 9 shows the performance metrics for each model category.

Figure 19

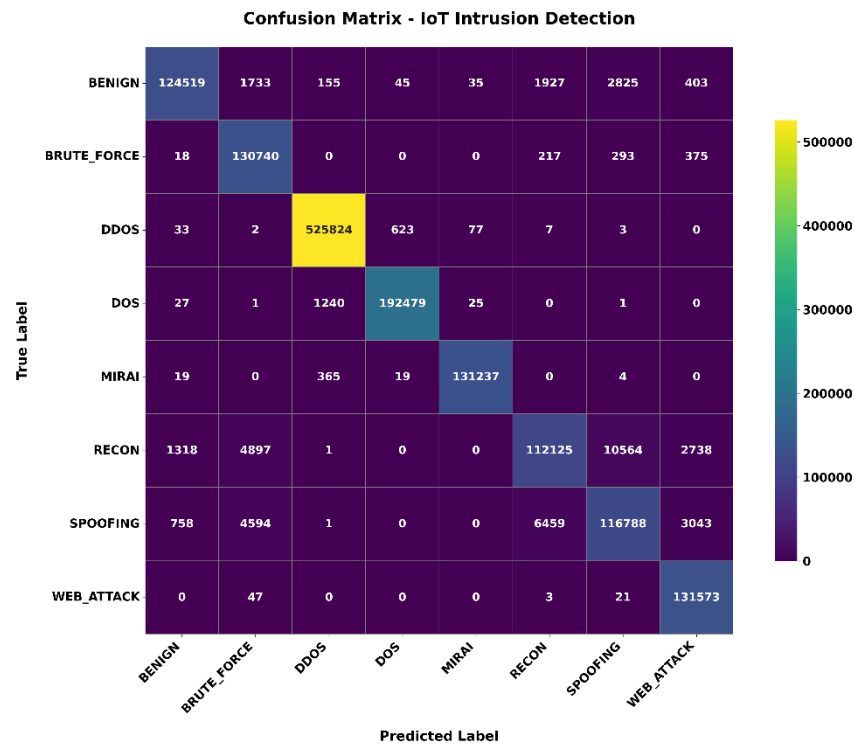


Figure 20

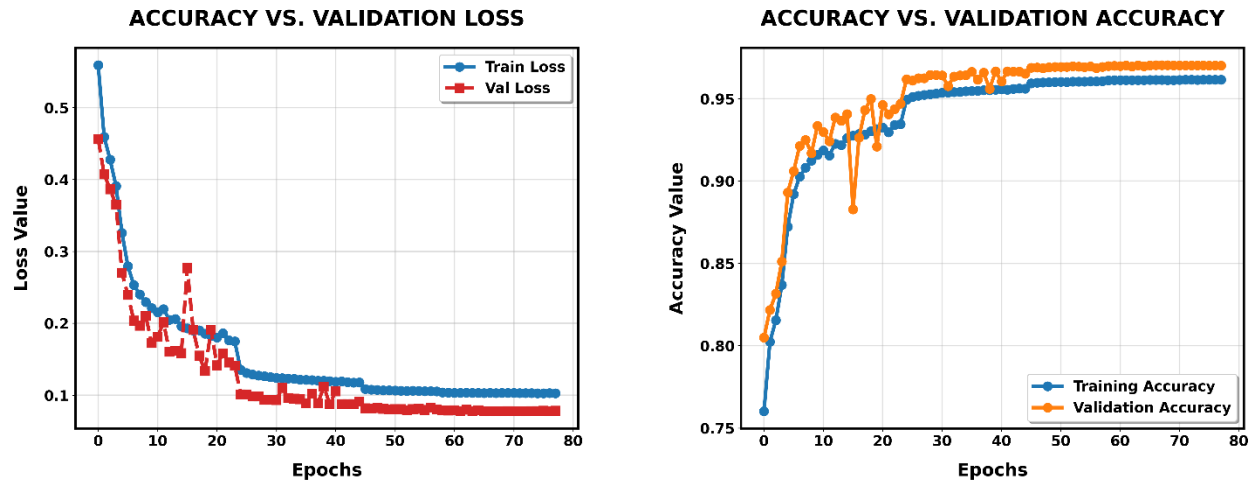


Figure 21

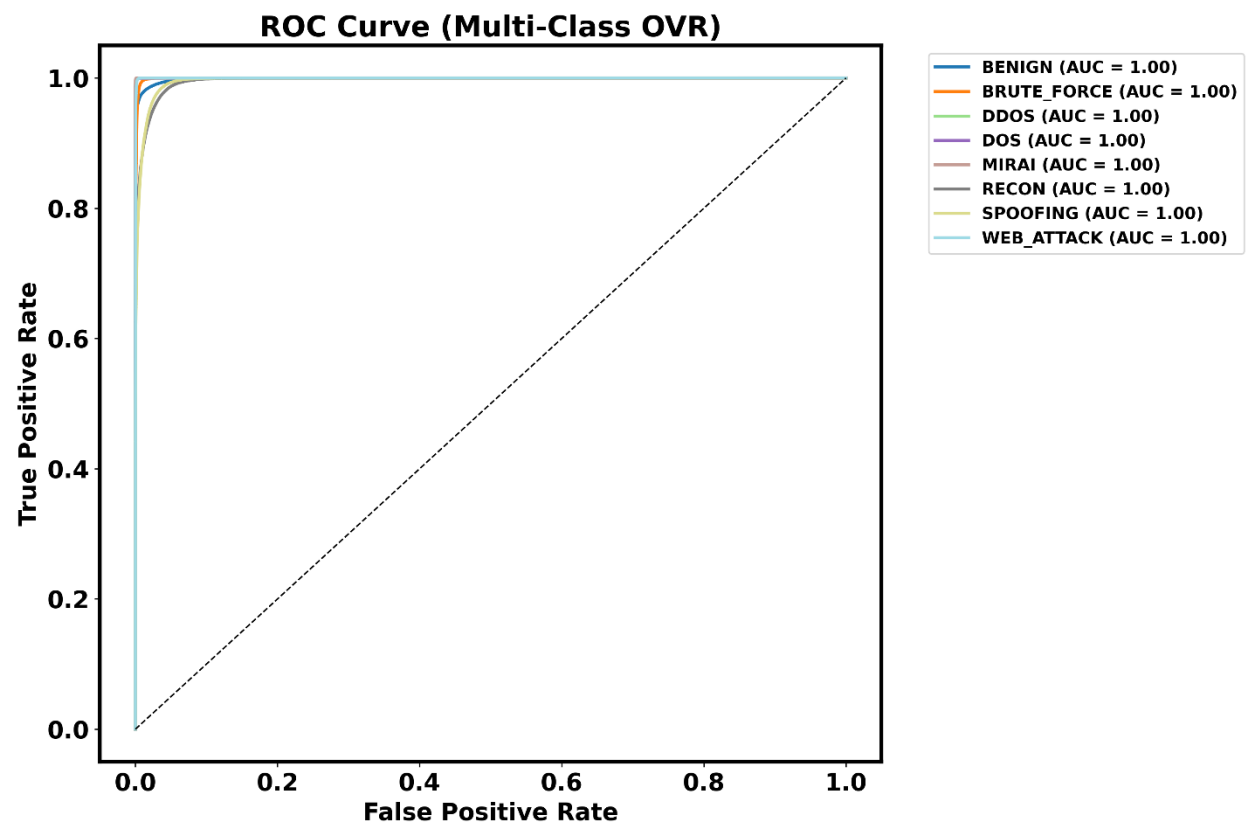


Figure 22

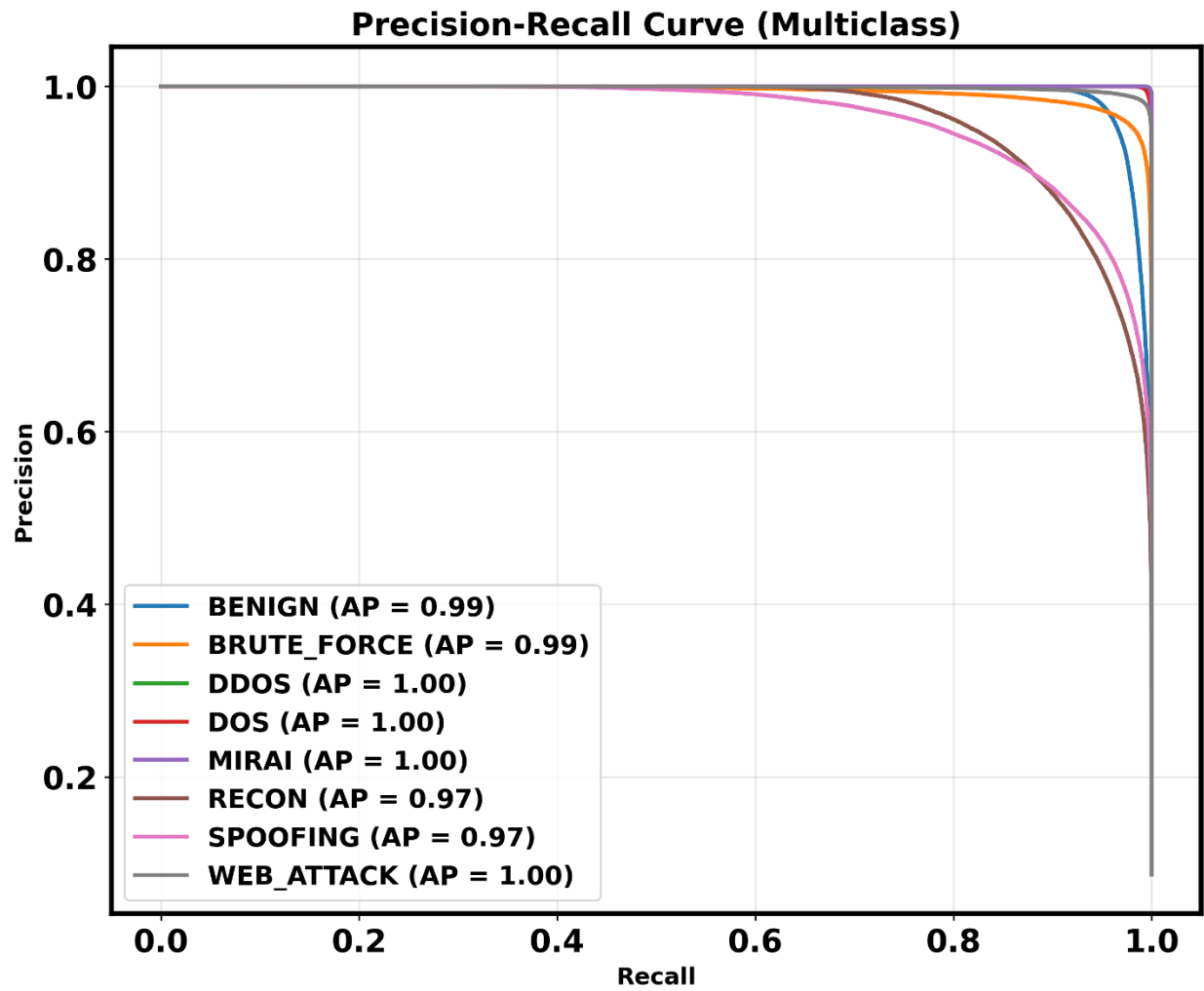


Table 8

Class	TP	FN	FP	TN
BENIGN	124519	7123	2173	1376386
BRUTE_FORCE	130740	903	11274	1367284
DDOS	525824	745	1762	981870
DOS	192479	1294	687	1315741
MIRAI	131237	407	137	1378420
RECON	112125	19518	8613	1369945
SPOOFING	116788	14855	13711	1364847
WEB_ATTACK	131573	71	6559	1371998

- True Positive (TP): Correctly predicted instances for the specific class.
- False Positive (FP): Incorrectly predicted as target class (Sum of Column - TP).
- False Negative (FN): Target class predicted as something else (Sum of Row - TP).
- True Negative (TN): Correctly predicted as NOT belonging to target class.

Table 9

Class	Acc(%)	Pr(%)	Rc(%)	F1(%)
BENIGN	99.38	98.28	94.59	96.40
BRUTE_FORCE	99.19	92.06	99.31	95.55
DDOS	99.83	99.67	99.86	99.76
DOS	99.87	99.64	99.33	99.49
MIRAI	99.96	99.90	99.69	99.79
RECON	98.14	92.87	85.17	88.85
SPOOFING	98.11	89.49	88.72	89.10
WEB_ATTACK	99.56	95.25	99.95	97.54

4.5.4 LSTM [11]

This model LSTM [11] performed well across most metrics but took longer to train. It achieved Accuracy 98.58%, Precision 98.02%, Recall 9.99%, F1-Score 98.00%, AUC-Score 99.96%, MCC-Score 98.27%, and Kapp-Score 98.27%. It also demonstrated balanced performance across all categories, as shown by the Confusion Matrix in Figure 23. Initially, its performance fluctuated between the Train-Accuracy and Validation-Accuracy indicators, but it stabilized after Epoch 40, as shown in Figure 24. The Roc-Curve and Precision-Recall curves are also illustrated in Figures 25 and 26, respectively, The model's Confusion Matrix calculations, which represent all positive and negative cases for each category, are shown in Table 10, and Table 11 also displays the performance metrics for each category of the model.

Figure 23

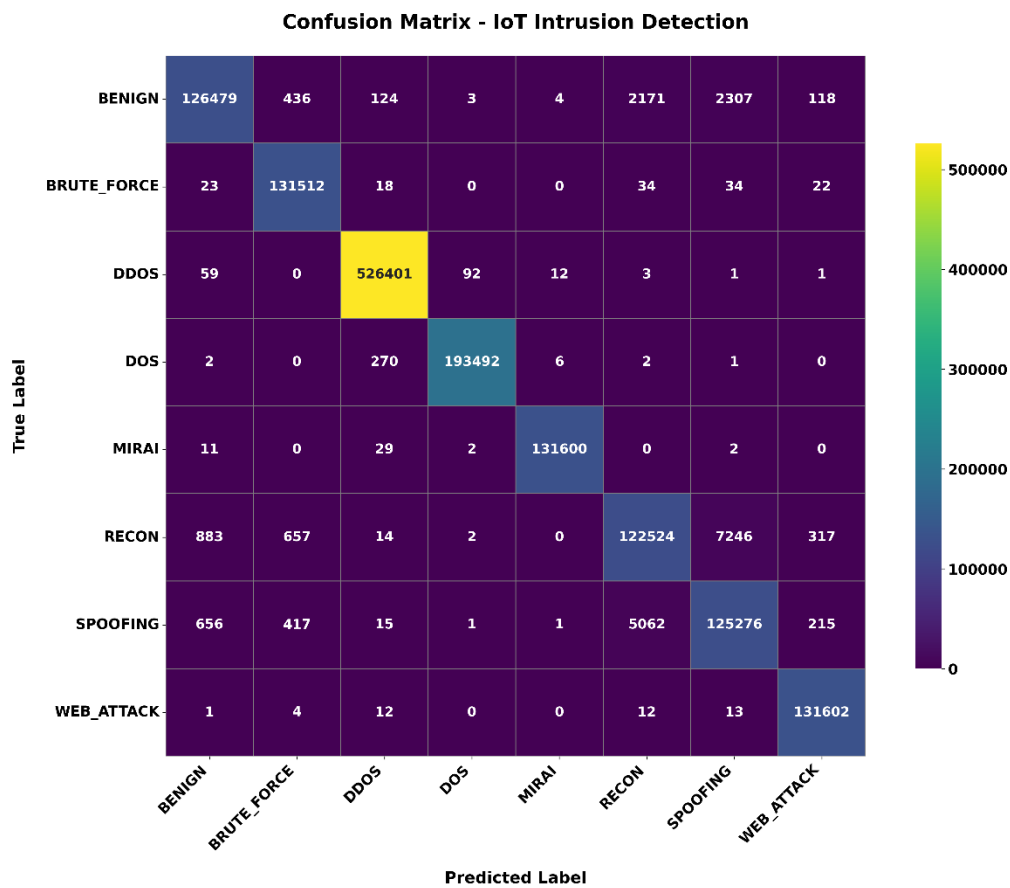


Figure 24

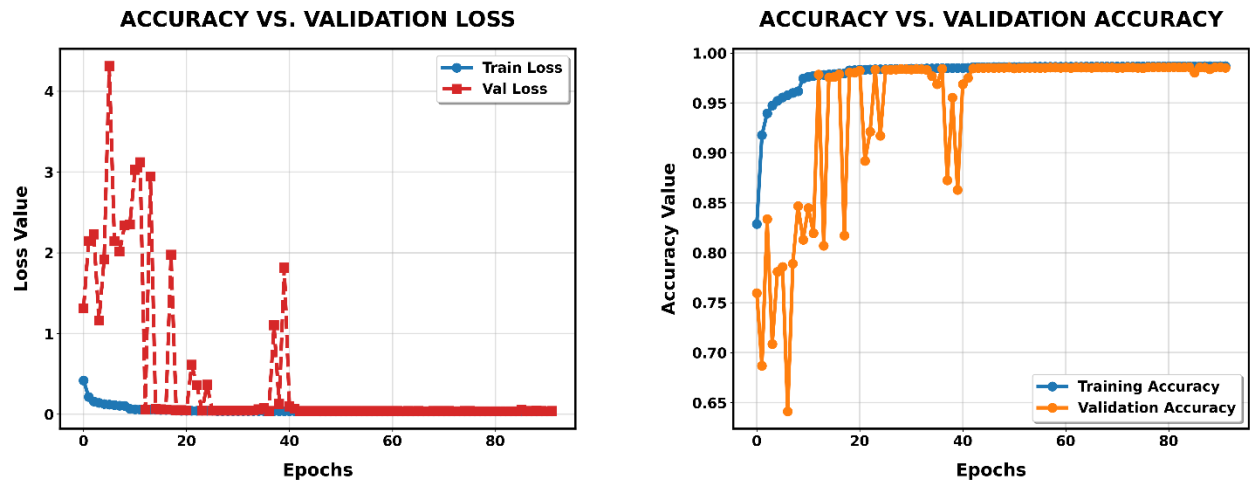


Figure 25

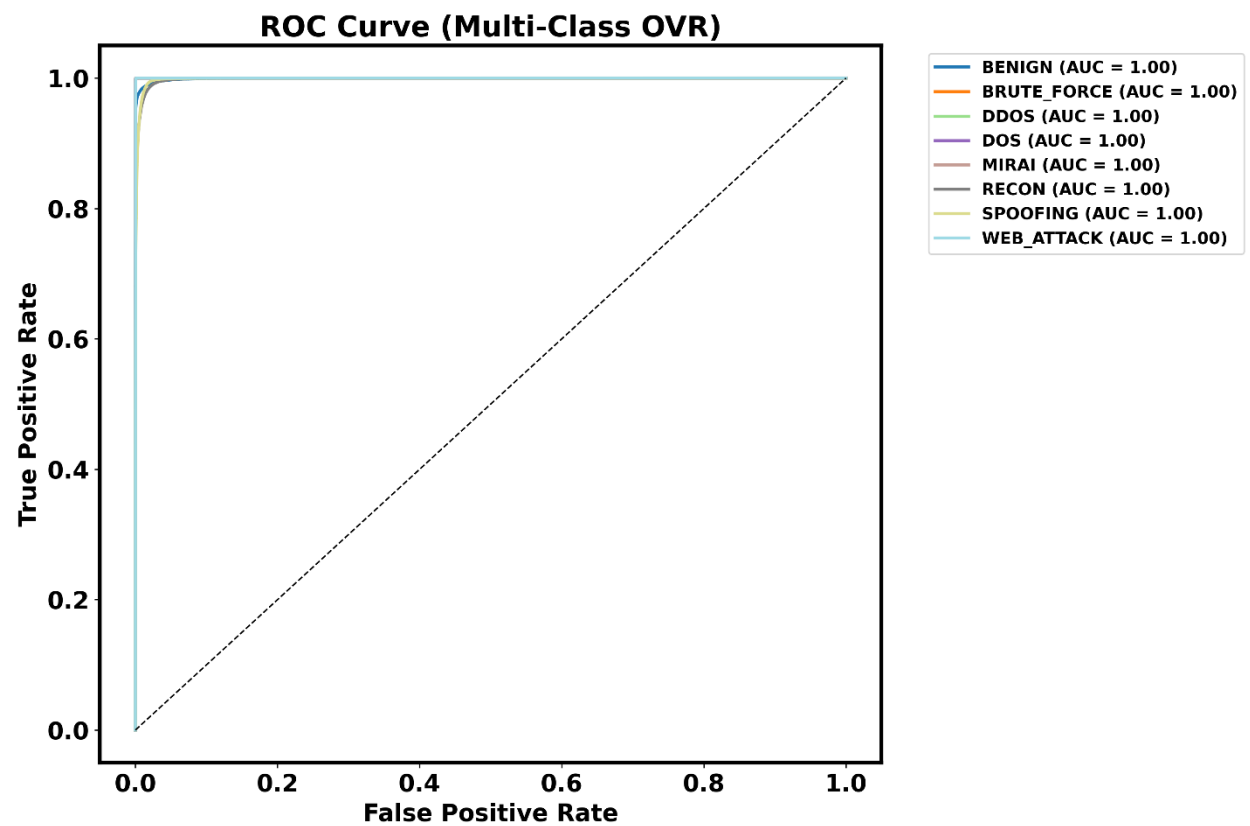


Figure 26

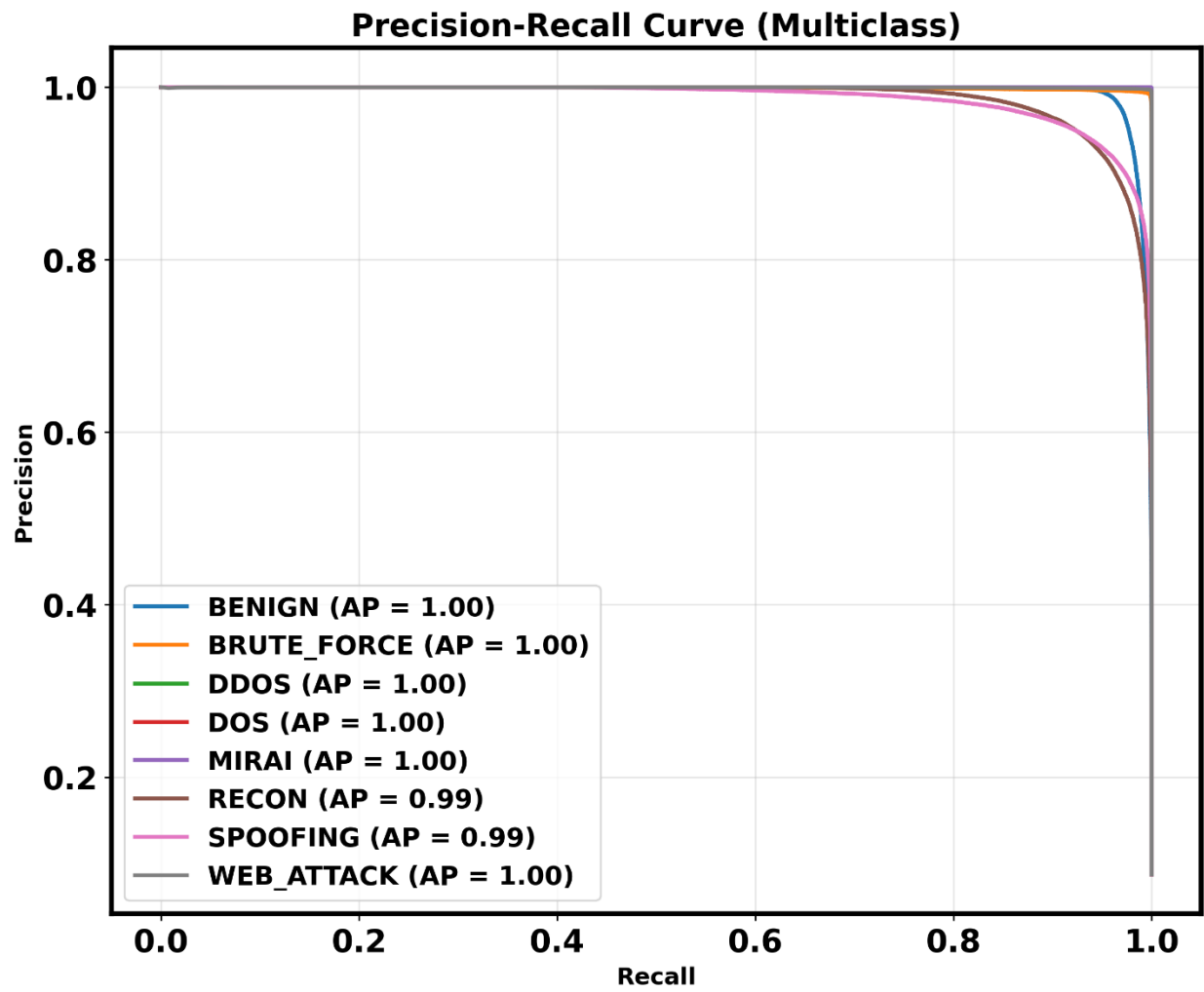


Table 10

Class	TP	FN	FP	TN
BENIGN	126479	5163	1635	1376924
BRUTE_FORCE	131512	131	1514	1377044
DDOS	526401	168	482	983150
DOS	193492	281	100	1316328
MIRAI	131600	44	23	1378534
RECON	122524	9119	7284	1371274
SPOOFING	125276	6367	9604	1368954
WEB_ATTACK	131602	42	673	1377884

- True Positive (TP): Correctly predicted instances for the specific class.
- False Positive (FP): Incorrectly predicted as target class (Sum of Column - TP).
- False Negative (FN): Target class predicted as something else (Sum of Row - TP).
- True Negative (TN): Correctly predicted as NOT belonging to target class.

Table 11

Class	Acc(%)	Pr(%)	Rc(%)	F1(%)
BENIGN	99.55	98.72	96.08	97.38
BRUTE_FORCE	99.89	98.86	99.90	99.38
DDOS	99.96	99.91	99.97	99.94
DOS	99.97	99.95	99.85	99.90
MIRAI	100.00	99.98	99.97	99.97
RECON	98.91	94.39	93.07	93.73
SPOOFING	98.94	92.88	95.16	94.01
WEB_ATTACK	99.95	99.49	99.97	99.73

4.5.5 CNN-BILSTM [6]

This CNN-BILSTM [6] model also achieved outstanding performance; while not the best, it was exceptional. It achieved Accuracy 98.14%, Precision 97.35%, Recall 97.36%, F1-Score 97.35%, AUC-Score 99.94%, MCC-Score 97.73%, and Kapp-Score 97.73%. The model performed exceptionally well in time-dependent attacks thanks to the BiLSTM [6] feature, which allows it to review time-dependent attacks from both directions. Figure 27 shows the model's Confusion Matrix, where we observe a balance in most categories. Figure 28 shows the model's performance during the learning phase, where we find some fluctuation at the beginning of the learning phase, followed by complete stability after Epoch 30, and near-perfect convergence between the Train-Accuracy and Validation-Accuracy indicators, We also analyzed the model's performance using the ROC-Curve and Precision-Curve curves, respectively, in Figures 29 and 30. Table 12 presents the Confusion Matrix calculations for all classes, and Table 13 shows the different performance metrics for each class.

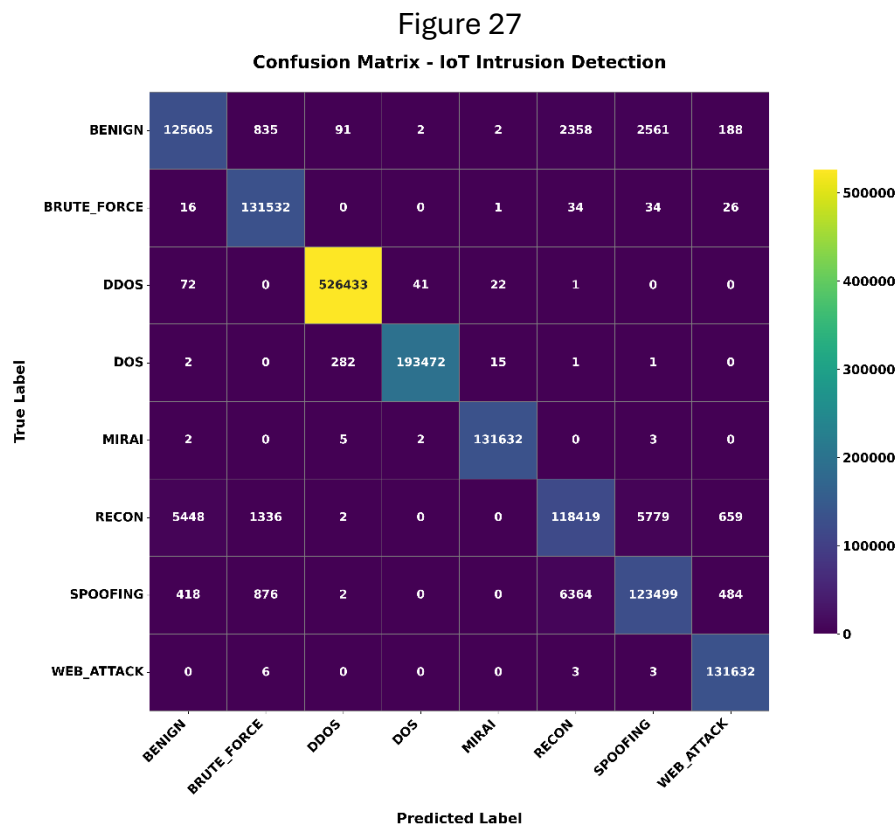


Figure 28

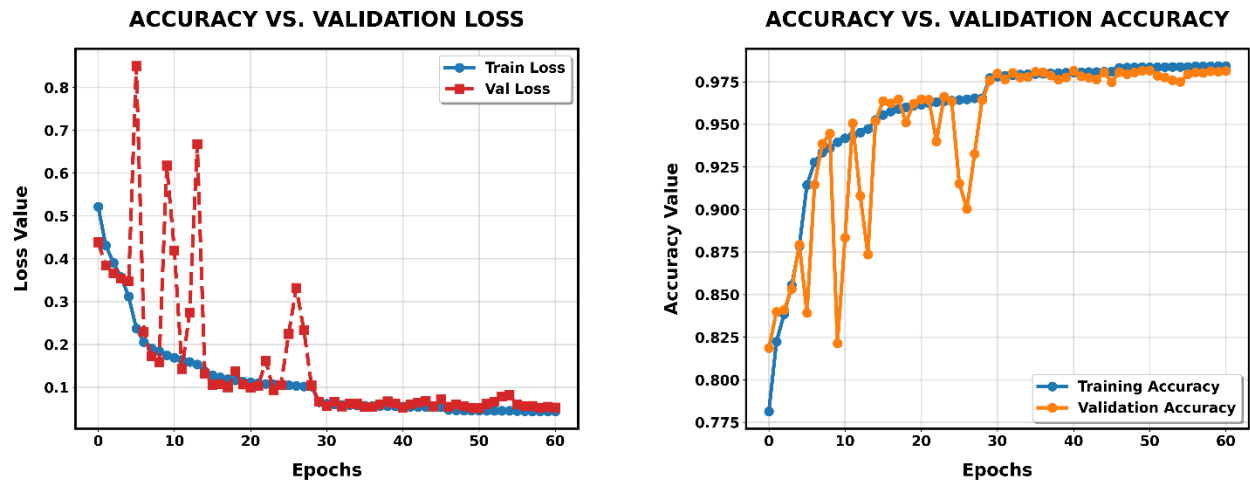


Figure 29

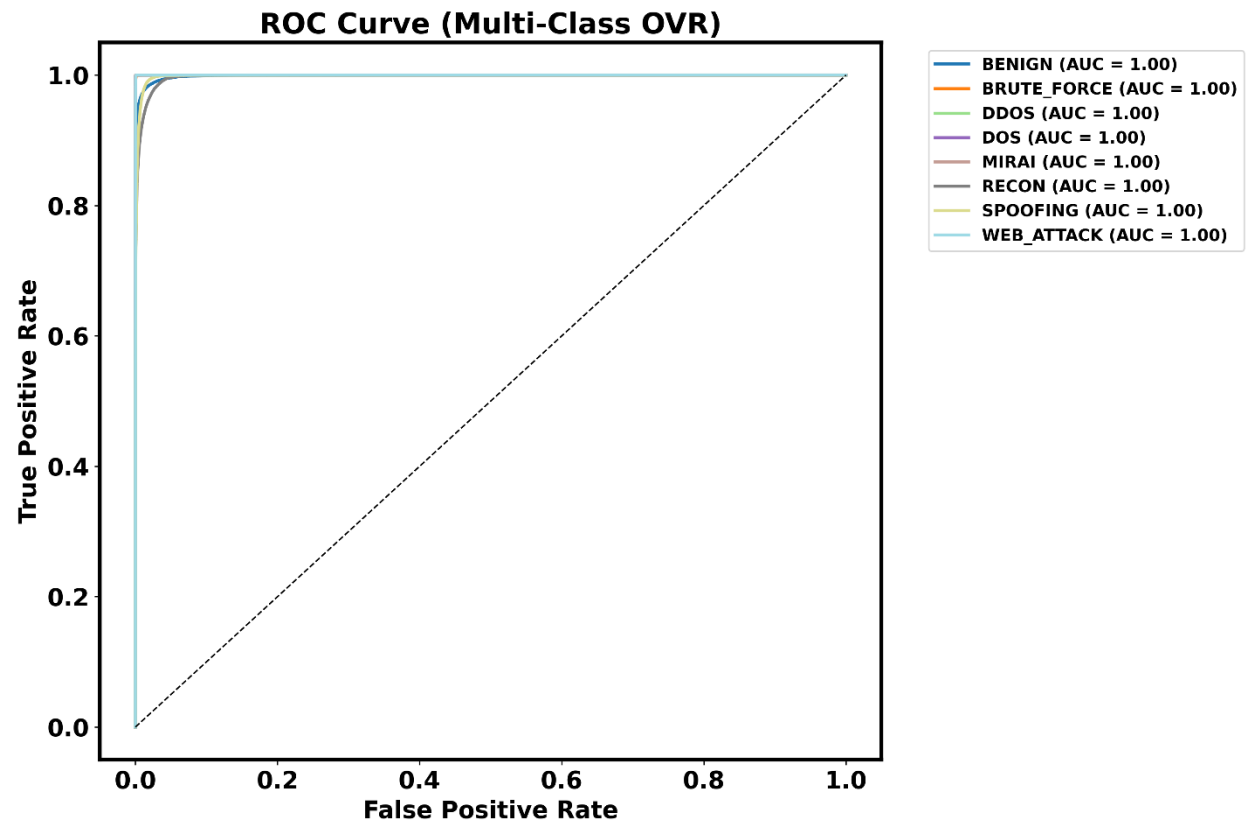


Figure 30

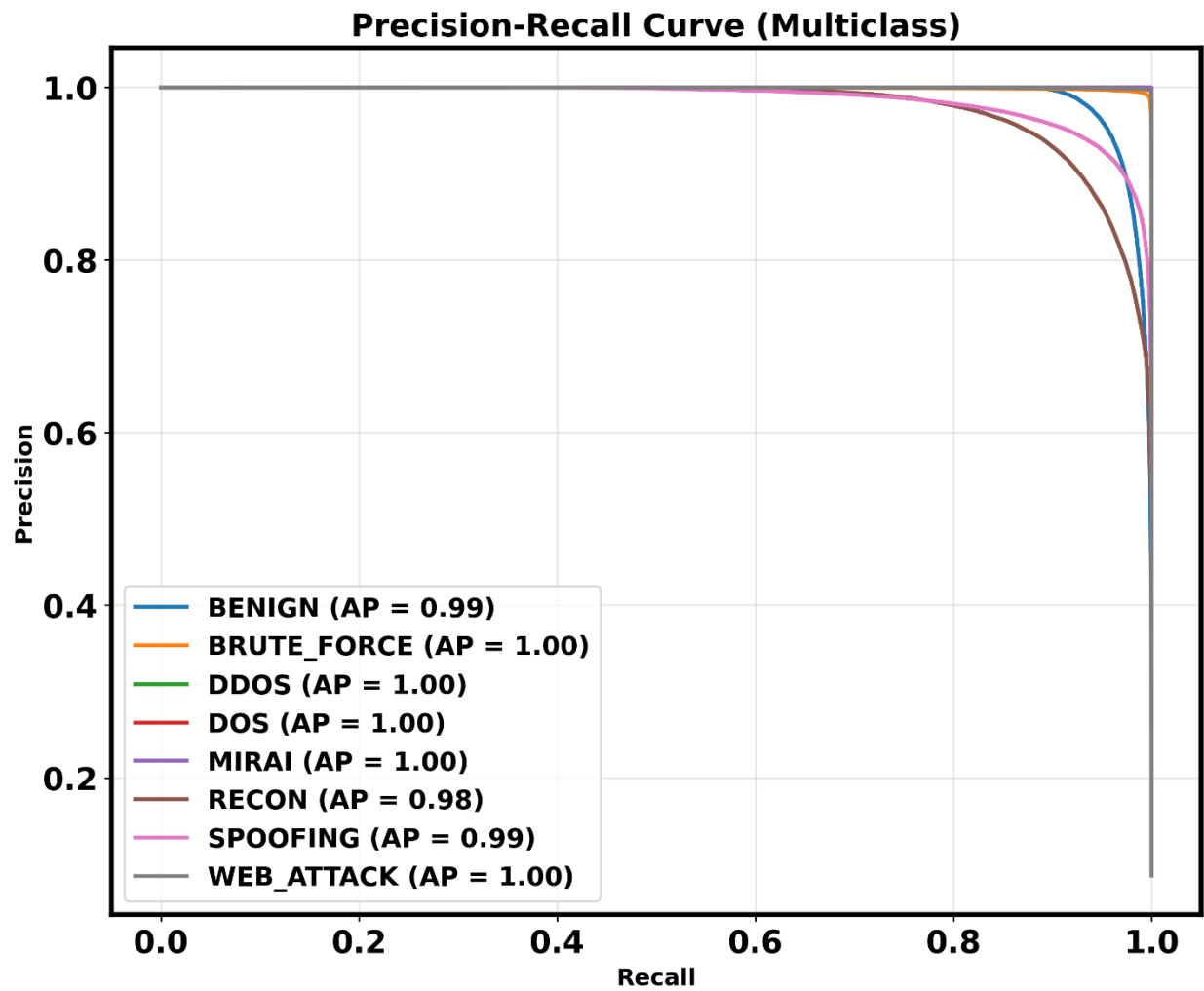


Table 12

Class	TP	FN	FP	TN
BENIGN	125605	6037	5958	1372601
BRUTE_FORCE	131532	111	3053	1375505
DDOS	526433	136	382	983250
DOS	193472	301	45	1316383
MIRAI	131632	12	40	1378517
RECON	118419	13224	8761	1369797
SPOOFING	123499	8144	8381	1370177
WEB_ATTACK	131632	12	1357	1377200

- True Positive (TP): Correctly predicted instances for the specific class.
- False Positive (FP): Incorrectly predicted as target class (Sum of Column - TP).
- False Negative (FN): Target class predicted as something else (Sum of Row - TP).
- True Negative (TN): Correctly predicted as NOT belonging to target class.

Table 13

Class	Acc(%)	Pr(%)	Rc(%)	F1(%)
BENIGN	99.21	95.47	95.41	95.44
BRUTE_FORCE	99.79	97.73	99.92	98.81
DDOS	99.97	99.93	99.97	99.95
DOS	99.98	99.98	99.84	99.91
MIRAI	100.00	99.97	99.99	99.98
RECON	98.54	93.11	89.95	91.51
SPOOFING	98.91	93.64	93.81	93.73
WEB_ATTACK	99.91	98.98	99.99	99.48

After completing the training of all models and compiling the results, we found that the LSTM [11] model is the best in terms of results and the most effective in exploring time patterns between volumes. However, this architecture is criticized for being the most time-consuming in terms of training, As shown in Table 14, the second best model in terms of results is the CAE-TCN [3] model, which also achieved outstanding results in exploring time patterns using the Sliding-Window feature. However, this model is considered the best among them in terms of the time taken in the training process, The worst performer among the five was CNN-LSTM [4], achieving 97.02 Accuracy and 95.89 Precision. This model also failed to demonstrate significant competitiveness in terms of training time, Therefore, based on the method we used in the model's data training process, which is the Sliding-Window method that includes 10 packets together and their pass-through to the model to enable it to detect time patterns, we find that the LSTM [11] architecture, in terms of results.

Table 14

Model	Acc(%)	Pr(%)	Rc(%)	F1(%)	AUC	MCC	Kappa
AE-LSTM-CNN [2]	98.44 %	97.80%	97.79%	97.6 %	99.95 %	98.10 %	98.10 %
CAE-TCN [3]	98.56 %	97.99 %	97.96 %	97.9 %	99.9 %	98.2 %	98.2 %
CNN-LSTM [4]	97.02 %	95.89 %	95.82 %	95.81 %	99.88 %	96.36 %	96.36 %
LSTM [11]	98.58 %	98.02 %	97.99 %	98.00 %	99.96 %	98.27 %	98.27 %
CNN-BILSTM [6]	98.14 %	97.35 %	97.36 %	97.35 %	99.94 %	97.73 %	97.73 %

References

- [1] R. Chinnasamy, M. Subramanian, N. Sengupta, and R. MP, "Contextual Internet of Things Intrusion Detection: A Sliding Window Convolutional Neural Network–Gated Recurrent Unit Model Enhanced by Graph Neural Networks," *Cureus Journals*, vol. 2, no. 1, 2025.
- [2] B. Susilo, A. Muis, and R. F. Sari, "Intelligent intrusion detection system against various attacks based on a hybrid deep learning algorithm," *Sensors*, vol. 25, no. 2, p. 580, 2025.
- [3] H. Wang, Y. Yang, and P. Tan, "CTWA: a novel incremental deep learning-based intrusion detection method for the Internet of Things," *Artif Intell Rev*, vol. 58, no. 12, p. 374, Oct. 2025, doi: 10.1007/s10462-025-11358-9.
- [4] A. Halbouni, T. S. Gunawan, M. H. Habaebi, M. Halbouni, M. Kartiwi, and R. Ahmad, "CNN-LSTM: Hybrid Deep Neural Network for Network Intrusion Detection System," *IEEE Access*, vol. 10, pp. 99837–99849, 2022, doi: 10.1109/ACCESS.2022.3206425.
- [5] Md. A. Hossain, "Deep Learning-Based Intrusion Detection for IoT Networks: A Scalable and Efficient Approach," Mar. 26, 2025, *In Review*. doi: 10.21203/rs.3.rs-6042512/v1.
- [6] M. Zahid and T. S. Bharati, "Enhancing cybersecurity in IoT systems: a hybrid deep learning approach for real-time attack detection," *Discov Internet Things*, vol. 5, no. 1, p. 73, Jul. 2025, doi: 10.1007/s43926-025-00156-y.
- [7] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.
- [8] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning," *Journal of machine learning research*, vol. 18, no. 17, pp. 1–5, 2017.
- [9] A. Fernández, S. Garcia, F. Herrera, and N. V. Chawla, "SMOTE for learning from imbalanced data: progress and challenges, marking the 15-year anniversary," *Journal of artificial intelligence research*, vol. 61, pp. 863–905, 2018.
- [10] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [11] M. A. Hossain, "Deep learning-based intrusion detection for IoT networks: a scalable and efficient approach," *EURASIP Journal on Information Security*, vol. 2025, no. 1, p. 28, 2025.

- [12] D. Park, S. Kim, H. Kwon, D. Shin, and D. Shin, "Host-Based Intrusion Detection Model Using Siamese Network," *IEEE Access*, vol. 9, pp. 76614–76623, 2021, doi: 10.1109/ACCESS.2021.3082160.
- [13] E. C. P. Neto, S. Iqbal, S. Buffett, M. Sultana, and A. Taylor, "Deep learning for intrusion detection in emerging technologies: a comprehensive survey and new perspectives," *Artif Intell Rev*, vol. 58, no. 11, p. 340, Aug. 2025, doi: 10.1007/s10462-025-11346-z.
- [14] F. S. Alsubaei, "Smart deep learning model for enhanced IoT intrusion detection," *Sci Rep*, vol. 15, no. 1, p. 20577, Jul. 2025, doi: 10.1038/s41598-025-06363-5.
- [15] W. Wu, H. Fouzi, S. Sidi-Mohammed, and S. Ying, "Improving cyber-attack detection in Internet of Medical Things using ensemble deep learning methods," *Cluster Comput*, vol. 28, no. 14, p. 891, Nov. 2025, doi: 10.1007/s10586-025-05660-y.
- [16] W. Wu, H. Fouzi, B. Benamar, S. Sidi-Mohammed, and S. Ying, "Deep learning-based stacked models for cyber-attack detection in industrial internet of things," *Neural Comput & Applic*, vol. 37, no. 24, pp. 19617–19651, Aug. 2025, doi: 10.1007/s00521-025-11418-9.
- [17] A. Okutan Kara, M. Kara, and A. Boyaci, "A Comparative Analysis of Machine Learning and Deep Reinforcement Learning Approaches for Adaptive Intrusion Detection," *IEEE Access*, vol. 13, pp. 189833–189849, 2025, doi: 10.1109/ACCESS.2025.3627098.
- [18] M. A. Ali and S. A. H. Al-Sharafi, "Intrusion detection in IoT networks using machine learning and deep learning approaches for MitM attack mitigation," *Discov Internet Things*, vol. 5, no. 1, p. 48, Apr. 2025, doi: 10.1007/s43926-025-00104-w.
- [19] C. K. K. Reddy *et al.*, "Twined ensemble framework for network security: integrating Random Forest, AdaBoost, and Gradient Boosting for enhanced intrusion detection," *Discov Internet Things*, vol. 5, no. 1, p. 107, Oct. 2025, doi: 10.1007/s43926-025-00199-1.
- [20] A. K. Shukla, S. Dwivedi, and A. Mishra, "An effective hybrid deep learning metaheuristic model for robust IoT intrusion detection," *Discov Computing*, vol. 28, no. 1, p. 200, Sep. 2025, doi: 10.1007/s10791-025-09708-w.
- [21] K. Qiu, M. Yan, T. Luo, and F. Chen, "FedAware: a distributed IoT intrusion detection method based on fractal shrinking autoencoder," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 37, no. 7, p. 202, Sep. 2025, doi: 10.1007/s44443-025-00153-9.
- [22] B. Buyuktanir, Ş. Altinkaya, G. Karatas Baydogmus, and K. Yildiz, "Federated learning in intrusion detection: advancements, applications, and future directions," *Cluster Comput*, vol. 28, no. 7, p. 473, Sep. 2025, doi: 10.1007/s10586-025-05325-w.
- [23] P. Selvam *et al.*, "Federated learning-based hybrid convolutional recurrent neural network for multi-class intrusion detection in IoT networks," *Discov Internet Things*, vol. 5, no. 1, p. 39, Apr. 2025, doi: 10.1007/s43926-025-00130-8.
- [24] B. Susilo, A. Muis, and R. F. Sari, "Intelligent Intrusion Detection System Against Various Attacks Based on a Hybrid Deep Learning Algorithm," *Sensors*, vol. 25, no. 2, p. 580, Jan. 2025, doi: 10.3390/s25020580.
- [25] M. Sajid *et al.*, "Enhancing intrusion detection: a hybrid machine and deep learning approach," *J Cloud Comp*, vol. 13, no. 1, p. 123, Jul. 2024, doi: 10.1186/s13677-024-00685-x.

- [26] A. Halbouni, T. S. Gunawan, M. H. Habaebi, M. Halbouni, M. Kartiwi, and R. Ahmad, "CNN-LSTM: Hybrid Deep Neural Network for Network Intrusion Detection System," *IEEE Access*, vol. 10, pp. 99837–99849, 2022, doi: 10.1109/ACCESS.2022.3206425.
- [27] F. Pedregosa *et al.*, "Scikit-learn: machine learning in python, journal of machine learning research. vol 12 (Oct)," 2011.
- [28] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of machine learning research*, vol. 9, no. Nov, pp. 2579–2605, 2008.